

HTML et CSS, les bases

Savoir quand choisir le HTML5

La norme HTML5 permet d'utiliser de nouvelles fonctionnalités, de nouvelles balises, de nouvelles API, mais elle n'est pas disponible partout. En effet, il s'agit d'une norme récente, et seuls les navigateurs les plus récents la supporte de manière acceptable, aucun ne la supportant de manière complète à ce jour.

Afin de savoir si votre navigateur supporte efficacement le norme ou une fonctionnalité particulière, certains sites et outils sont à votre disposition notamment:

- <http://caniuse.com>
- <http://html5test.com>
- <http://html5readiness.com>
- Etc...

Utiliser HTML5 avec des navigateurs non modernes

Il est possible d'utiliser partiellement HTML5 avec des navigateurs qui ne le supportent pas de base à l'aide de bibliothèques Javascript telles que:

- html5shiv: <https://github.com/aFarkas/html5shiv>
- html5shim: <http://html5shim.googlecode.com/svn/trunk/html5.js>
- html5boilerplate: <http://html5boilerplate.com/>
- Etc...

... Ou vous pouvez vous-même créer votre propre script Javascript qui introduit les nouvelles entités HTML5 au sein de votre navigateur en vous basant sur la flexibilité de la norme XHTML (héritée de la norme XML).

Notions de base du HTML

Le balisage en HTML5 se présente de la même façon que dans les versions précédentes, à ceci près qu'il n'y a qu'une seule et unique bonne façon de faire.

Les noms des balises se composent uniquement de caractères en minuscules (camel case interdit), allant de A à Z, les chiffres sont interdits

On ouvre un élément en l'entourant des caractères '<' et '>' comme dans l'exemple suivant: <toto>. Puis on ferme ce même élément comme dans l'exemple suivant: </toto>. Tout le contenu se situant entre ces deux balises sera considéré comme étant un descendant (direct ou indirect) de l'élément.

Il existe des balises appelées 'auto-fermantes', elles se présentent comme dans l'exemple suivant: <toto/>.

Doctype, NameSpace, Encoding

La structure de base d'une page HTML se compose d'un noeud racine (root), le noeud **<html>**. Au dessus de ce noeud racine se trouve ce que l'on appelle un DOCTYPE ... Qu'est-ce qu'un DOCTYPE ?

Le DOCTYPE, ou déclaration de type de documents, est un fichier qui sert à définir les éléments qui vont se trouver à l'intérieur de notre document HTML. Le navigateur pourra donc interpréter correctement les éléments fournis et vérifier leur intégrité.

Durant toute l'histoire de la norme (X)HTML, il y a plusieurs DOCTYPE la norme HTML5 peut se vanter d'avoir simplifié le processus, aujourd'hui il n'existe plus qu'un seul DOCTYPE universel (on peut l'utiliser avec les vieilles normes (X)HTML par exemple) et le voici: `<!doctype html>`

Chaque document utilise de un à plusieurs namespaces, parfois de manière explicite, souvent de manière implicite. Un namespace est un mécanisme intégré à la norme XML qui permet d'éviter des conflits de noms, en effet, lorsque deux documents veulent utiliser le même nom de balise, ils peuvent chacun déclarer leur propre namespace, ainsi ils évitent de rentrer en collision sur les noms de balises qu'ils peuvent partager et définir des sens et utilisations différentes pour chacune.

Chaque document utilise d'ailleurs un jeu de caractères, un encodage précis, depuis plusieurs années, il est courant d'encoder les fichiers au format UTF-8.

Les nouveaux éléments

La norme HTML5 intègre sont lot de nouvelles balises, certaines reliées à une API spécifique (comme la balise canvas) ou livrées avec une sémantique particulière (article)

- La balise section
- La balise article
- La balise aside
- La balise nav
- La baise header
- La balise footer
- La balise command
- La balise details
- La balise summary
- La balise canvas
- Etc...

Compatibilité entre navigateurs

Le HTML5 est une norme assez jeune, par conséquent aucun navigateur ne la supporte pleinement. Il est fréquent en revanche de trouver parmi les dernières versions, des navigateurs supportant la majorité des fonctionnalités intéressantes de la norme, voici un petit tableau donnant un aperçu des navigateurs supportant le mieux la norme au moment où ce cours est rédigé (mars 2017).

BROWSERS

Select up to five browsers and compare their test results in detail

492 Chrome 52	474 Firefox 53	312 Internet Explorer 11	489 Opera 37	419 Safari 10.2
------------------	-------------------	-----------------------------	-----------------	--------------------

parsing rules

	5	5	5	5	5
<!DOCTYPE html> triggers standards mode	Yes ✓	Yes ✓	Yes ✓	Yes ✓	Yes ✓
HTML5 tokenizer	Yes ✓	Yes ✓	Yes ✓	Yes ✓	Yes ✓
HTML5 tree building	Yes ✓	Yes ✓	Yes ✓	Yes ✓	Yes ✓
Parsing inline SVG	Yes ✓	Yes ✓	Yes ✓	Yes ✓	Yes ✓
Parsing inline MathML	Yes ✓	Yes ✓	Yes ✓	Yes ✓	Yes ✓

LES FORMULAIRES HTML5

La balise form en HTML5

La balise form est une balise qui existe depuis les premières versions de la norme HTML, elle permet de créer une interface utilisateur permettant à l'internaute de communiquer certaines données. Il est également possible de soumettre ce formulaire et de créer une requête HTTP utilisant le verbe GET ou POST (voir la norme HTTP 1.1 pour plus de précisions).

La norme HTML5 introduit son lot de nouveautés et nous permet de mieux cibler les besoins du développeur et des utilisateurs quand à l'utilisation des formulaires.

Créer et utiliser des formulaires HTML5

Pour créer un formulaire, il suffit d'utiliser la balise `<form>` comme dans l'exemple suivant:

```
<form method="GET" action="index.html">  
  <label>Nom: </label>  
  <input type="text" name="username" />  
</form>
```

champ de type date

Le champ de type Date permet d'activer une aide au remplissage (type datepicker) présente uniquement sur quelques navigateurs et différente de l'un à l'autre. Le contenu attendu est une date du calendrier Grégorien au format RFC3339 sans précision de la timezone, mais le champ accepte un contenu vide ou mal formaté sans retourner d'erreur.

La liste des attributs compatibles est la suivante :

- name, disabled, form, type
- autocomplete, autofocus, list, min, max
- step (chiffre entier), readonly, required, value, pattern
- attributs classiques d'événements et xml.

```
<form method="GET" action="index.html">
  <input type="date" max="2026-07-23" min="1986-07-23" name="the_date" />
</form>
```

champ de type datetime

Ce type de champ est une combinaison des deux type date et time. Il permet donc de renseigner une date et une heure précise. Les valeurs renvoyées par ce type de champ de formulaire sont différentes selon le navigateur et la précision n'est pas la même (de la seconde au centième de seconde suivant le navigateur).

```
<form method="GET" action="index.html">  
  <input type="datetime" name="my_datetime">  
</form>
```

Champ de type time

Ce type de champ permet de renseigner une heure, avec plus ou moins de précision. Le format attendu est le même que pour le champ de type date, sans localisation (timezone). Ce type de champ déclenche l'affichage d'une sorte de sélecteur permettant d'incrémenter ou décrémenter la valeur du champ minute par minute.

- name, disabled, form, type
- autocomplete, autofocus, list, min, max
- step (interface & float), readonly, required, value, pattern
- attributs classiques d'événements et xml.

```
<form method="GET" action="index.html">  
  <input type="time" name="wakeup">  
</form>
```

Champ de type url

En apparence ce champ ressemble à celui de type text. Cependant, le navigateur attend cette fois un format bien spécifique devant respecter un pattern de type url, tous les types d'URL sont admis (ftp://, mailto:, http://, etc.).

```
<form method="GET" action="index.html">  
  <input type="url" value="http://" name="my_website" />  
</form>
```

Champ de type email

Ce champ est équivalent au type url, seul le format attendu change. Il attend au minimum un caractère (caractère non accentué comprenant les séparateurs tiret ou underscore) suivi d'un @ suivi à son tour d'un caractère.

```
<form method="GET" action="index.html">  
  <input type="email" name="user_email" />  
</form>
```

L'attribut autofocus

Si l'on veut qu'un focus soit appliqué directement sur l'un des inputs d'une page web il faut lui ajouter l'instruction autofocus. Pour s'assurer que l'autofocus soit pris en compte, le mieux est de lui donner la valeur 'autofocus'.

Voir exercice: [html](#)

pattern & validation

Les champs requis non complétés ou ne respectant pas une certaine règle de syntaxe définie par le type du formulaire, ou par l'attribut pattern bloquent le processus d'envoi des données d'un formulaire.

Cette validation à la volée permet à l'utilisateur d'être informé très rapidement de ses erreurs et de les corriger étape par étape grâce aux indications fournies par les infobulles situées sous chacun des champs invalides.

Des patterns par défaut existent pour certains types de champ, comme les champs de type email ou url par exemple, qui attendent des formats spécifiques

D'autres patterns - ou masques - peuvent être créés sur-mesure, grâce à l'attribut pattern qui accepte une expression régulière.

```
<form method="GET" action="index.html">
  <!--
    Attend une valeur numérique d'au moins 5 chiffres précédés
    d'une lettre majuscule comprise entre A et F.
  -->
  <input type="text" pattern="[A-F][0-9]{5}" />
</form>
```

AUDIO ET VIDEO EN HTML5

Les balises <audio> et <video>

La balise <audio> permet comme son nom l'indique, d'intégrer un fichier son à la page HTML, en d'autres termes, elle définit un flux ou streaming audio.

La balise <video> permet d'intégrer et de lire des fichiers vidéos au sein d'une page HTML. Avant son arrivée, il fallait à la place utiliser un plugin pour lire des vidéos, comme Adobe Flash, d'ailleurs Celui-ci reste en partie utilisé pour regarder des vidéos sur Youtube, Dailymotion etc... Mais utiliser un plugin a de nombreux défauts :

- On dépend de ceux qui gèrent le plugin (l'entreprise Adobe possède Flash Player)
- On ne peut pas toujours contrôler son fonctionnement
- Il y a parfois des failles de sécurité

Au final, c'est assez lourd c'est pour cela que deux nouvelles balises standard ont été créées en HTML5 :<video> et <audio>!

La balise <video>

Il est possible d'ajouter des attributs aux balises video afin de modifier leur comportement. Vous pouvez compléter la balise des attributs suivants :

- poster: image à afficher à la place de la vidéo tant que celle-ci n'est pas lancée. Par défaut, le navigateur prend la première image de la vidéo.
- controls: pour ajouter les boutons « Lecture », « Pause » et la barre de défilement.
- width: pour modifier la largeur de la vidéo, les proportions sont conservées.
- height: pour modifier la hauteur de la vidéo, les proportions sont conservées.
- loop: la vidéo sera jouée en boucle.
- autoplay: lance la lecture de la vidéo dès le chargement de la page.
- preload: indique si la vidéo peut être préchargée dès le chargement de la page ou non. On ne peut pas forcer le préchargement de la vidéo, c'est toujours le navigateur qui décide.

```
<!--  
  Le message sera affiché si le navigateur ne prend pas en charge la balise <audio>  
-->  
<video src="uncharted.mp4" controls poster="uncharted.jpg" autoplay>  
  Veuillez mettre à jour votre navigateur !  
</video>
```

La balise <audio>

Il est possible d'ajouter des attributs aux balises audio et video afin de modifier leur comportement. Vous pouvez compléter la balise des attributs suivants :

- controls: pour ajouter les boutons « Lecture », « Pause » et la barre de défilement
- autoplay: la musique sera jouée dès le chargement de la page. Évitez d'en abuser, c'est en général irritant d'arriver sur un site qui joue de la musique tout seul !
- loop: la musique sera jouée en boucle.
- width: pour modifier la largeur de l'outil de lecture audio.
- preload: indique si la musique peut être préchargée dès le chargement de la page ou non. On ne peut pas forcer le préchargement de la musique, c'est toujours le navigateur qui décide.

```
<!--  
  Le message sera affiché si le navigateur ne prend pas en charge la balise <audio>  
-->  
<audio src="sound.mp3" width="100px" autoplay loop preload controls>  
  Veuillez mettre à jour votre navigateur !  
</audio>
```

Formats des fichiers audio

Il existe plusieurs formats de fichiers sons, selon le navigateur utilisé, il faudra utiliser l'un ou l'autre parmi les suivants:

- MP3: C'est l'un des plus vieux, mais aussi l'un des plus compatibles (tous les appareils savent lire des MP3), ce qui fait qu'il est toujours très utilisé aujourd'hui.
- AAC: utilisé majoritairement par Apple sur iTunes, c'est un format de bonne qualité. Les iPod, iPhone et autres iPad savent les lire sans problème.
- OGG: le format Ogg Vorbis est très répandu dans le monde du logiciel libre, notamment sous Linux. Ce format a l'avantage d'être libre, c'est-à-dire qu'il n'est protégé par aucun brevet.
- WAV (format non compressé) : évitez autant que possible de l'utiliser car le fichier est très volumineux avec ce format. C'est un peu l'équivalent du Bitmap (BMP) pour l'audio.

La compatibilité dépend des navigateurs, mais elle évolue dans le bon sens au fil du temps. Pensez à consulter Canibase.com pour savoir en temps réel quel navigateur supporte quel format.

Formats des fichiers video

Pour le stockage de la vidéo, on a besoin de trois éléments.

Un format conteneur : c'est un peu comme une boîte qui va servir à contenir les deux éléments ci-dessous. On reconnaît en général le type de conteneur à l'extension du fichier : AVI, MP4, MKV...

Un codec audio : c'est le format du son de la vidéo (MP3, AAC, OGG...)

Un codec vidéo : c'est le format qui va compresser les images, les principaux à connaître pour le Web sont :

- H.264 : l'un des plus puissants et des plus utilisés aujourd'hui mais il n'est pas 100% gratuit.
- Ogg Theora : un codec gratuit et libre de droits, mais moins puissant que H.264. Il est bien reconnu sous Linux mais, sous Windows, il faut installer des programmes pour pouvoir le lire.
- WebM : un autre codec gratuit et libre de droits, plus récent. Proposé par Google, c'est le concurrent le plus sérieux de H.264 à l'heure actuelle.

La compatibilité dépend des navigateurs, mais elle évolue dans le bon sens au fil du temps. Pensez à consulter Canius.com pour savoir en temps réel quel navigateur supporte quel format.

Gérer plusieurs sources vidéos/audios

Comment faire pour choisir le bon fichier vidéo alors que chaque navigateur supporte des formats différents ? Et bien sachez qu'il est tout à fait possible de spécifier plusieurs sources audios et videos à un même élément !

```
<!-- plusieurs sources audio -->
<audio controls>
  <source src="uncharted.mp3">
  <source src="uncharted.ogg">
</audio>

<!-- plusieurs sources video -->
<video controls poster="uncharted.jpg" width="600">
  <source src="uncharted.mp4">
  <source src="uncharted.webm">
  <source src="uncharted.ogv">
</video>
```

VUE D'ENSEMBLE DES APIS POUR HTML5

Les APIs HTML5

La norme HTML5 apporte son lot de nouvelles fonctionnalités, de balises etc ... Parmi ces nouveautés, on peut compter de nouvelles fonctionnalités accessibles en Javascript et regroupées en API (Application Programming interface). Voici un petit tour d'horizon (non exhaustif) de ces API:

- File API: Permet de gérer des fichiers côté client.
- Geolocation API: Permet d'avoir des renseignements sur la position de l'utilisateur.
- Fullscreen API: Permet de passer le navigateur en mode plein écran.
- getUserMedia API: Permet de capturer du son ou de la vidéo via le matériel de l'internaute.
- Battery Status API: Permet d'accéder à l'état de la batterie.
- Web Storage API: Permet de stocker localement des données.
- Offline web app API: Permet de garantir le fonctionnement d'une app, même hors ligne.
- Web Workers et Web Messaging API: Mutli-threading & communication avec le programme principal via Web Messaging.
- Notifications API: Permet d'afficher des notifications de bureau à l'utilisateur.
- Canvas API: Permet de définir une zone de dessin 2D ou 3D.
- Server-Sent Events API: Permet de recevoir des informations du serveur.
- History API: Permet de gérer et maîtriser l'historique de navigation.
- Drag & Drop API: Support natif du Drag & Drop.
- ... Et bien d'autres comme Web Speech API, Web Telephony API etc ...

Gestion des fichiers

HTML5 fournit des API très puissantes permettant d'interagir avec les données binaires et le système de fichiers local d'un utilisateur. La suite d'API File offre aux applications Web la possibilité de:

- Créer des objets Blob arbitraires
- Lire des fichiers de façon synchrone ou asynchrone
- Créer des objets Blob arbitraires
- Ecrire des fichiers dans un emplacement temporaire
- Lire un répertoire de fichiers de façon récursive
- Effectuer des opérations de glisser-déposer de fichiers depuis le Bureau vers le navigateur
- Importer des données binaires à l'aide de XMLHttpRequest2.

L'objet FileList

Un objet de ce type est retourné par la propriété `files`, ce qui vous permet d'accéder à la liste des fichiers sélectionnés avec l'élément `<input type="file" />` ou en utilisant l'API de glisser-déposer à l'aide de l'objet `DataTransfer`

Ci-dessous, un exemple de récupération d'une `FileList` initialisée par l'utilisation de la balise `input` mise à disposition de l'utilisateur.

```
<script type="text/javascript">

function fileHandler(event)
{
    //récupère une liste de fichiers de type FileList
    var files = event.target.files;
    var i = files.length;

    while( --i > -1 )
    {
        // files[i] est un objet de type File
        console.log(files[i].name);
    }
}
</script>

<!-- l'attribut multiple permet à l'utilisateur de sélectionner plusieurs fichiers à la fois-->
<input type="file" id="input" multiple onchange="fileHandler">
```

L'objet File

L'objet File fournit un tableau contenant des informations sur l'accès au contenu du ou des fichiers sélectionnés, des informations sur le type mime la taille le nom des fichiers ainsi que le nombre de fichiers sélectionnés.

Ils sont généralement extraits d'un objet `FileList` retournée comme résultat lors de la sélection des fichiers en utilisant un input de type file, ou à partir d'un objet `DataTransfer` lors d'un glisser-déposer.

Cet objet expose 3 propriétés standards:

- **name**: **DOMString** le nom du fichier courant. lecture seul.
- **size**: **unsigned long** la taille en octets du fichier. lecture seul.
- **type**: **DOMString** le type (MIME type) du fichier. lecture seul.

```
var files = event.target.files;
var i = files.length;
var myfile = null;

while (--i > -1) {
    myfile = files[i];
    // affichage des informations concernant le fichier
    console.log(myfile.name, myfile.size, myfile.type);
}
```

Drag & Drop API

L'objet FileReader permet, de manière asynchrone, de lire le contenu des fichiers (ou des tampons de données brutes) stockés sur l'ordinateur de l'utilisateur en utilisant des objets File ou Blob. Pour créer un FileReader, il suffit de faire comme ceci:

```
function fileHandler(event) {
    //récupère une liste de fichiers de type FileList
    var files = event.target.files;
    var i = files.length;
    var reader = null;

    while (--i > -1) {

        // création d'un nouvel objet de type FileReader
        reader = new FileReader();
        // quand le fichier sera lu et le contenu disponible, fileHandler sera invoquée.
        reader.addEventListener("loadend", fileHandler);
        // readAsDataURL récupère le contenu du fichier encodé en base64
        reader.readAsDataURL(files[i]);
    }
}

function fileDataHandler(event) {
    // récupère le contenu du fichier encodé en base64
    var base64Data = event.target.result;
}
```

Drag & Drop API

Vous pouvez aussi autoriser l'utilisateur à glisser-déposer (drag and drop) des fichiers dans votre application web. La première étape consiste définir une zone de dépôt. Pour se faire, il suffit de demander à un élément de votre page d'écouter et de gérer certains événements:

```
// cette fonction de callback annule le comportement par défaut du navigateur
// ( ouverture des fichiers ou rafraichissement de la page )
function cancelHandler(event) {
    event.stopPropagation();
    event.preventDefault();
}

function dropHandler(event) {
    e.stopPropagation();
    e.preventDefault();
    // on peut récupérer le tableau de fichier droppé par l'utilisateur à l'aide
    // de la propriété dataTransfer de l'objet event
    var files = event.dataTransfer.files;
}

// ici dropbox est l'élément html sur lequel l'utilisateur va dropper ses fichiers
var dropbox = document.getElementById("dropbox");
dropbox.addEventListener("dragenter", cancelHandler, false);
dropbox.addEventListener("dragover", cancelHandler, false);
dropbox.addEventListener("drop", dropHandler, false);
```

Site web hors ligne

HTML5 a introduit pas mal de nouvelles fonctionnalités au sein de ses API pour permettre aux développeurs de proposer des versions offline de leurs applications. En effet, il n'est pas toujours possible pour les utilisateurs (surtout les mobinautes) de disposer en permanence d'une connexion internet, or il est souhaitable, surtout de nos jours, qu'une application continue de fonctionner même offline.

A la charge du développeur de prendre cela en compte et de pousser les modifs en ligne une fois le réseau revenu. Voici la liste des nouveautés potentiellement intéressantes:

- Application Cache: permet de mettre en cache des fichiers sur le disque dur de l'utilisateur et de s'en servir en cas d'absence de connexion internet.
- LocalStorage: permet de stocker de façon permanente (si on le souhaite), des données sur le disque utilisateur (une sorte de super cookie)
- IndexedDB: permet d'avoir au sein du navigateur une base de données basée sur des paires clés/valeurs au format JSON
- WebSQL: permet d'avoir au sein du navigateur une BDD de type SQLite

Application Cache

HTML5 supporte la mise en cache hors-ligne de ressource (fichiers), stockés dans l'Application Cache (AppCache), référencés dans un fichier manifest (*.appcache), ce qui permet les bénéfices suivants :

- Navigation hors-ligne: les ressource sont accessibles hors réseau.
- Vitesse : les ressources mises en cache sont locales, et se chargent donc plus rapidement.
- Réduction de la charge serveur : Update des ressources qui ont changées sur le serveur uniquement.

```
<html manifest="my_manifest.appcache">  
  <!-- code html -->  
</html>
```

CACHE MANIFEST

```
# mis en cache au premier telechargement  
style.css  
image.png
```

```
# pages de repli en cas de ressource inaccessible  
FALLBACK:  
index.html fallback.html
```

```
# fichiers qui contourne l'appcache  
NETWORK:  
network.html
```

Session & Local Storage

L'API Storage donne la possibilité au développeur de stocker des informations de 2 façons différentes. La première avec la propriété sessionStorage de l'objet window, qui va stocker les données tant que l'utilisateur ne ferme pas le navigateur

La seconde, localStorage, fonctionne exactement de la même manière mais va stocker les données sans délai d'expiration, sur le disque de l'utilisateur.

```
// stocke une donnée dont la clé est "name" et la
// valeur "Dark Vador" en session
sessionStorage.setItem("name", "Dark Vador");

// récupérer la valeur de session et l'affiche dans la console
console.log(sessionStorage.getItem("name"));

// stocke une donnée dont la clé est "realname" et la
// valeur "Anakin Skywalker" de façon permanente
localStorage.setItem("realname", "Anakin Skywalker Vador");

// récupérer la valeur permanente et l'affiche dans la console
console.log(localStorage.getItem("realname"));
```

Historiques de pages

L'objet DOM window fournit un accès à l'historique du navigateur via l'objet history. Il expose un ensemble de méthodes et de propriétés qui permettent d'avancer et de reculer dans l'historique de l'utilisateur ainsi que de manipuler le contenu de l'ensemble de l'historique (HTML5 only).

```
// avance et recule d'un cran dans l'historique
window.history.back();
window.history.forward();

// avance et recule de 5 crans d'un coup dans l'historique
window.history.go(-5);
window.history.go(5);

// longueur de l'historique ...
console.log(window.history.length);

// pousse une nouvelle url au sein de l'historique & la page poussée est ouverte.
// La méthode "replaceState()" fait exactement la même chose ? ceci prouve qu'elle
// remplace l'état courant.
var stateObj = { foo: "bar" };
history.pushState(stateObj, "page 2", "bar.html");

// donne l'état courant
console.log(window.history.state);
```

CSS3 : LES NOUVEAUTES

Mise en page et positionnement

Là où les anciennes versions de la norme CSS nous proposaient un positionnement 'inline', 'block', 'flottant', la norme CSS3 nous apporte son lot de nouveautés en la matière. Il est en effet possible d'utiliser maintenant un nouveau mode de positionnement pour le modèle classique, j'ai nommé le 'display-block', qui a pour avantage pouvoir positionner des blocks ... les uns à la suite des autres et non pas les uns en dessous des autres.

Mais ce n'est pas tout, il existe maintenant un nouveau modèle de positionnement en CSS3, le 'flexbox layout'. Ce mode de positionnement permet de placer des éléments un peu à la manière d'un photoshop ou d'un éditeur wysiwyg. Il est donc dorénavant possible:

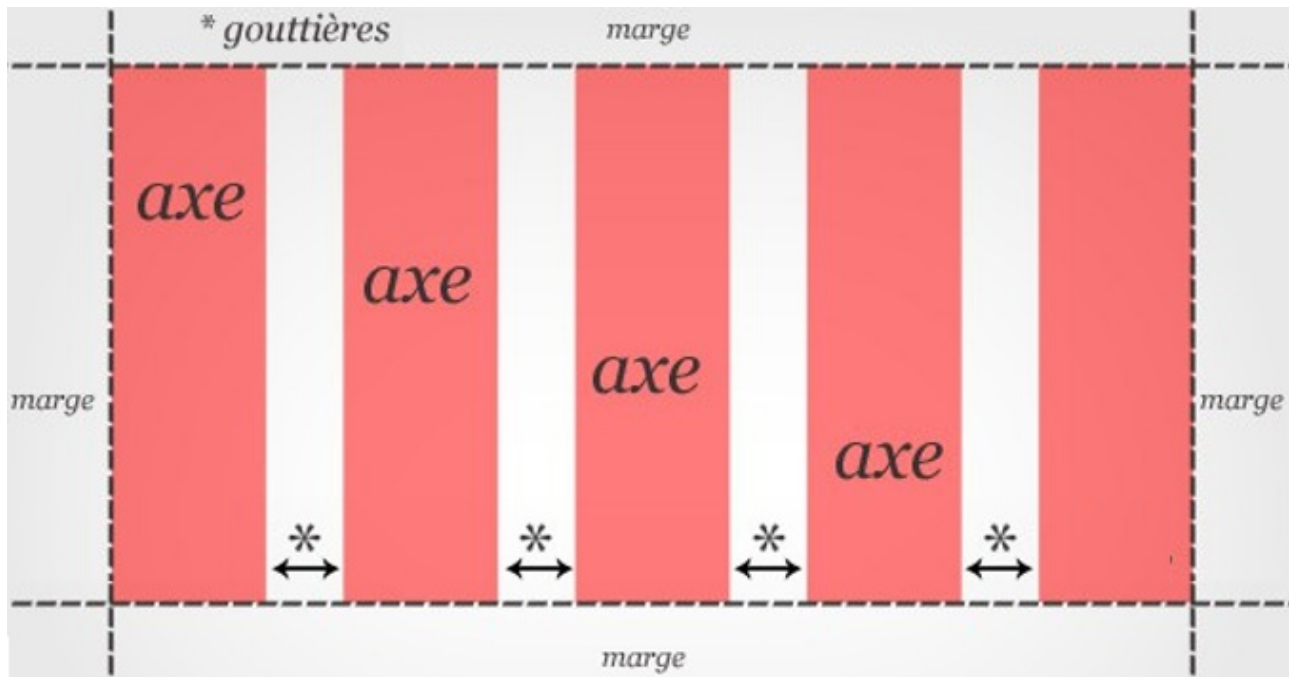
- De définir un coefficient de contraction / étirement des éléments
- De centrer verticalement et horizontalement
- De définir un axe principal et un axe secondaire de positionnement
- De remanier l'ordre d'apparition des éléments
- Etc...

```
.container{
  /*Tout les éléments enfants deviennent des flex-items*/
  display: flex;

  /* les éléments apparaissent en colonne inversée*/
  flex-direction: column-reverse;
}
```

Positionnement par grille

Le concept général de Grid Layout (ou 'positionnement en grille') est de diviser virtuellement l'espace en lignes et en colonnes. Une grille est constituée de repères (axes) verticaux et/ou horizontaux, séparés par des gouttières permettant de structurer le contenu. Il ne faut pas oublier les marges externes (pour les éventuels bords de la fenêtre) et internes (de chaque axe).



```
.row {  
  width : 100%;  
  margin-left: 2%;  
}
```

```
.row > div {  
  float: left;  
  margin : 1px 1px;  
  height: 100px;  
}
```

```
.w-1 { width : 8%; }  
.w-2 { width : 16%; }  
.w-3 { width : 24%; }  
.w-4 { width : 32%; }  
.w-5 { width : 40%; }  
.w-6 { width : 48%; }  
.w-7 { width : 56%; }  
.w-8 { width : 64%; }  
.w-9 { width : 72%; }  
.w-10 { width : 80%; }  
.w-11 { width : 88%; }  
.w-12 { width : 96%; }
```

Transformations (2D et 3D)

CSS3 apporte les transformations en 2 dimensions à travers la propriété transform et une liste de fonctions prédéfinies. La propriété CSS transform permet de manipuler un élément HTML sur les axes X et Y (horizontal et vertical) grâce à des fonctions diverses de transformation graphique. Il est donc possible de modifier l'apparence d'un élément grâce à un ensemble fonctions 2D :

- translate: la translation
- rotate: la rotation
- scale: mise à l'échelle
- skew: inclinaison
- matrix: matrice de transformation 3x3

Pour pouvoir appliquer des transformations, nous avons besoin de savoir quel est le point d'origine (d'ancrage) de la transformation. La propriété transform-origin définit ce point d'origine, sa valeur initiale est le centre de l'élément.

```
.my_elements{  
  /*centre de l'élément, valeur par défaut*/  
  transform-origin: 50% 50%;  
  
  /*décale l'élément de 50 pixels vers la droite et vers le bas*/  
  transform: translate(50,50);  
}
```

Transitions

Il est possible de réaliser des transitions entre deux valeurs pour une, ou plusieurs propriété(s) en CSS3. Le principe de base d'une transition CSS3 est de permettre une transition douce entre l'ancienne valeur et la nouvelle valeur d'une propriété CSS lorsqu'un événement est déclenché, comme un rollover par exemple.

La nouvelle directive css3 transition-property va recueillir les propriétés sur lesquelles on doit appliquer une transition.

La directive transition-duration va déterminer le temps (en secondes) pendant lequel la transition va opérer.

```
.my_elements{
  background-color: green;
  color: white;
}

.my_elements:hover{
  transition-property: background-color, color;
  transition-duration: 1s;
  color: #333333;
  background-color: #fafafa;
}
```

PREMIERS PAS AVEC JAVASCRIPT

Insérer du code JavaScript dans une page (X)HTML

Il y a 3 façons d'ajouter du code JavaScript dans une page:

- Directement dans les balises HTML, à l'aide d'un gestionnaire d'événements.
- En liant depuis la page HTML un fichier javascript externe contenant le code.
- En ajoutant le code JavaScript à l'intérieur de l'élément script.

```
<script type="text/javascript" src="js_file.js">
</script>

<!-- or -->

<script type="text/javascript">
  // js instructions there...
</script>
```

Traditionnellement il était d'usage de placer la balise `<script>` entre les tags `<head>` et `</head>`, cependant il est actuellement recommandé de la placer en fin de document juste avant `</body>` pour ne pas bloquer le chargement de la page, et exécuter les scripts uniquement lorsque le DOM est prêt.

Attention, la balise 'script' n'est pas auto-fermante, c'est-à-dire que l'on ne peut pas écrire `<script type="text/javascript" ... />`

L'attribut type doit être renseigné avec la syntaxe XHTML, mais il peut être omis en HTML5.

Les caractéristiques du Javascript

Le Javascript est un langage de haut niveau dérivé du C, à ce titre il possède plusieurs éléments de syntaxe proches de ce dernier tout en se distinguant sur d'autres. Il faut retenir avant tout les points suivants(liste non exhaustive):

- Javascript permet d'interagir avec le document HTML et le style des éléments.
- Il s'agit d'un langage exécuté avant tout côté client.
- Il peut écouter et réagir aux événements.
- C'est un langage orienté objet par prototypage.
- Les variables n'ont pas de type défini
- C'est un langage compilé à la volée, à l'aide d'un J.I.T (Just In Time Compiler).
- Les instructions sont séparées par des points-virgules, leur usage est optionnel.
- Les commentaires `//` peuvent commenter une seule ligne.
- Les commentaires `/* */` peuvent plusieurs lignes.
- Le mode non strict est le mode par défaut.
- Utiliser l'instruction `'use strict'`; en début de fichier permet de passer en mode strict.

Les différents types de données

Il existe différents types de données en Javascript ainsi qu'une multitude d'objets disponibles à travers certaines APIs ben spécifiques. Voici la liste des types de données de base:

- Nombres : int / float
- Chaînes de caractères (string) : encadrées de guillemets (simples ou doubles)
- Booléens (boolean) : false / true.
- Null (pas de valeur).
- undefined: absence de valeur définie.
- Function: les fonctions sont aussi des objets en Javascript.
- Objets : standard (Object)
- Tableau: (Array)
- Date (Date)
- Etc...

Manipulation du DOM

Le Document Object Model (DOM) est un standard du W3C pour hiérarchiser le contenu d'un document XML ou HTML. Une page HTML est constituée d'une série de nœuds, à l'aide de la DOM API de javascript, il est possible de manipuler ces nœuds.

Le DOM est interprété différemment en fonction des navigateurs il est donc parfois nécessaire d'adapter le code. Pour manipuler le DOM, on utilise l'objet document, disponible dans l'espace global.

```
// ici on cherche une référence à un élément par le biais de son attribut 'id'
var my_element = document.getElementById("element_id");

// ici on va chercher tout les éléments <div> d'un document HTML
var my_divs = document.getElementsByTagName("div");

// pour les navigateurs plus récents, il est possible d'utiliser les sélecteurs CSS
// ici, on va chercher tout les éléments disposant de la classe "code"
var my_elements = document.querySelectorAll(".code");

// il est possible de créer des nœuds...
var my_div = document.createElement("div");

// d'en insérer à l'intérieur d'autres nœuds...
my_div.appendChild(document.createElement("div"));

// de supprimer des enfants d'un nœud (ici my_node est une référence à un élément).
my_div.removeChild(my_node)
```

Les outils

Javascript n'est pas un langage précompilé, cela veut dire que si une erreur doit se produire, elle se produira lors de l'exécution du script, lorsque le J.I.T (Just In Time compiler) passera dans la portion de code "coupable".

Il existe cependant quelques outils qui peuvent faciliter la vie du développeur, en laissant la possibilité de debugger de manière efficace un script Javascript, en voici une liste non exhaustive:

- Console des différents navigateurs: (les meilleures sont celles de Chrome et de Firefox)
- L'environnement Node.js: ... Met à notre disposition une multitude de modules pour nous aider à debugger sous Node.
- L'instruction 'debugger': à mettre dans un script, nous permet de poser un breakpoint dans le code, on peut donc vérifier à tout moment, en ouvrant le console, l'état de chaque variable du programme à un instant T.
- Les méta langages tels TypeScript, qui viennent avec une palette d'outils comme un compilateur et un transpiler TypeScript -> Javascript ES5 & ES6. Ces langages sont fournis, précis, outillés et nous permettent d'obtenir du Javascript propre en sortie.

LE LANGAGE JAVASCRIPT

Déclaration et portée des variables

Javascript permet de stocker des données à l'intérieur de structures appelées variables. Ces dernières peuvent contenir tout type de valeur et ne sont pas limitées dans leur changement, par exemple une variable contenant une chaîne de caractères pourra stocker plus tard une valeur numérique (bien que ce ne soit pas une bonne pratique).

Les variables ont une portée, çàd qu'elles sont accessibles dans l'espace dans lequel elles ont été définies ainsi que dans tout les autres sous-espaces (ou espaces imbriqués de types closures etc ...).

La syntaxe pour définir une variable est: '**var ma_variable**'. Un nom de variable valide comporte tout les caractères alphanumériques hormis les caractères spéciaux et doit impérativement commencer par une lettre.

```
// définit une variable contenant la chaîne hello world
var msg = "hello world";

function hello() {
  // ici la variable est accessible car elle a été définie
  // dans l'espace global qui est un espace parent de la fonction
  alert(msg);
}
```

Les tableaux

Javascript nous mets à disposition des structures appelées **Arrays**, ou tableau. Ces tableaux permettent de stocker une suite de valeurs au lieu d'une seule. Il est possible de stocker des valeurs de type complètement différents les uns à la suite des autres. Pour déclarer un tableau, il existe deux types de syntaxes, la syntaxe objet et la syntaxe JSON.

Pour accéder à un élément de tableau, il suffit d'utiliser la syntaxe suivante: `tab[index]`, **ou index représente la position d'un élément à l'intérieur du tableau. Les tableaux sont '0-based'**, cela signifie que le 1er élément se trouve en position 0, le 2ème en position 1 etc...

```
// déclare un tableau vide ...
var tab1 = []; // syntaxe JSON
var tab2 = new Array(); // syntaxe objet

// déclare un tableau pré-rempli ...
var tab3 = [0, 80, "hello world"];

// affiche "hello world"
alert(tab3[2]);

// pousse un élément en 5ème position
alert(tab3[4]);
```

Opérateurs logiques et arithmétiques

Javascript introduit un certain nombre d'opérateur, un opérateur binaire nécessite deux opérandes, un opérateur unaire n'en nécessite qu'un seul, et un opérateur ternaire nécessite trois opérandes.

- Les opérateurs d'affectation
- Les opérateurs de comparaison
- Les opérateurs arithmétiques
- Les opérateurs binaires
- Les opérateurs logiques
- Les opérateurs de chaînes
- Les opérateurs ternaire (conditionnel)
- L'opérateur virgule
- L'opérateur unaire
- L'opérateur relationnels

```
"Les opérateurs d'affectation",  
("Les opérateurs de comparaison",  
"Les opérateurs arithmétiques",  
"Les opérateurs binaires",  
"Les opérateurs logiques",  
"Les opérateurs de chaînes",  
"Les opérateurs ternaire (conditionnel)",  
"L'opérateur virgule",  
"L'opérateur unaire",  
"L'opérateur relationnels")
```

```
//opérateurs arithmétiques:
```

```
alert(1 + 2); //addition  
alert(1 * 10); // multiplication  
alert(1 / 0.1); // division  
alert(10 - 5); // soustraction  
alert( 10 % 3 ); // modulo
```

```
// opérateurs logiques:
```

```
alert(10 == "10"); // égalité non stricte  
alert( 10 === "10"); // égalité stricte  
alert( 10 != "10" ); // non égalité non stricte  
alert( 10 !== "10" ); // non égalité stricte  
alert( (10==10) && (5==5)); // A est vrai et B est vrai  
alert( (10==10) || (5==5)); // A est vrai ou B est vrai
```

```
//ternaire;
```

```
alert( (10==8) ? "yes" : "no");
```

Les boucles

Les boucles permettent de répéter des actions simplement et rapidement, il y en a différents types mais elles se ressemblent toutes au sens où elles répètent une action un certain nombre de fois

```
var i = 0, prop = null, obj = {"name":"Dark Vador", "job":"Sith Lord" };
// boucle for
for (i = 0; i < 10; i++) {
  console.log(i);
}

// boucle while
i = 0;
while (i < 10) {
  console.log(i++);
}

// boucle do while
i = 0;
do {
  console.log(i++);
}
while (i < 10)

//boucle for in
for (prop in obj) {
  console.log(prop, obj[prop]);
}
```

Déclaration et appel de fonctions

Une fonction est une suite d'instructions que le développeur pourra utiliser (on dit invoquer) quand il en aura besoin. Elle peut avoir un nom, recevoir des paramètres, et retourner une valeur. Javascript apporte son propre lot de fonctions mais il est possible de coder les siennes !

```
// cette fonction nommée saySomething affiche une boîte de dialogue  
// contenant le message contenu dans le paramètre p_msg, elle fait appel  
// à une fonction Javascript nommée alert.
```

```
function saySomething(p_msg) {  
    alert(p_msg);  
}
```

```
// cette fonction ne porte pas de nom, on dit qu'elle est anonyme.  
// ici, elle est stockée dans une variable nommée myfunc  
var myfunc = function () {  
    alert("anonymous function");  
};
```

```
/// invoque la fonction saySomething et affiche une boîte de dialogue  
saySomething("Hello World");
```

```
// invoque la fonction anonyme à l'aide de la variable myfunc  
myfunc();
```

Gestion des erreurs et des exceptions

En Javascript, il est fréquent que des erreurs puissent survenir au sein du code, parfois elles n'ont (presque) aucune conséquence et le programme continue de fonctionner tant bien que mal. D'autres fois en revanche, il est tout à fait possible que le code plante complètement, soit à cause d'une étourderie du développeur (c'est plus fréquent qu'on ne le pense), soit parce qu'une erreur a été jeté volontairement.

Il est en effet possible d'interrompre volontairement le code en jetant un objet de type `Error` suite à un comportement prévu mais non voulu, pour cela il faut utiliser l'opérateur **throw**. Puisqu'il est possible de jeter des erreurs, il est possible de les attraper à l'aide du duo d'opérateurs **try ... catch**

```
// on essaie d'exécuter le code contenu dans le bloc try ...
try {
  // ... code potentiellement à risque.
}
catch (error) {
  // mais une erreur peut se produire et être jetée dans ce cas on l'attrape avec catch
  console.log(error);
}

// on peut également jeter nos propres erreurs à l'aide de l'opérateur throw
throw (new Error("This is an error, don't do that again !"));
```

LE JAVASCRIPT : UN LANGAGE ORIENTÉ OBJET

Développement Objet en JavaScript

JavaScript n'est pas un langage orienté objet classique, en effet son paradigme objet (sa façon de le penser et l'implémenter si vous préférez) n'est pas celui d'un langage orienté objet par classe.

En fait JavaScript est un **langage orienté objet par prototypage**, cela veut dire qu'il met l'accent sur la **décoration** d'objet, le **prototypage**, la **mutabilité** des objets. Dans un langage plus 'classique' comme Java, un objet aura un jeu de propriétés et de méthodes qui lui sont propres car définies sur une **classe**.

La notion de classe est absente en JavaScript ES5 (en ES6 il s'agit avant tout de 'sucre syntaxique'), à la place, le développeur va devoir **créer des objets et leur ajouter/supprimer/modifier des propriétés 'à la volée'**, c'est ça un langage orienté objet par prototypage !

Comme il n'y a pas de classe à proprement parler, **il n'y a pas** non plus de mécanisme propres aux langages orientés objet par classe, comme **l'héritage**, **l'interfacage**, le cloisonnage par **namespaces** etc... Si le développeur souhaite profiter de ce genre de mécanismes en JavaScript, il devra les **émuler à l'aide d'une bibliothèque** et/ou de fonctionnalités qu'il aura lui même créées.

En JavaScript, même les fonctions sont des objets, ce qui tombe bien car du coup elles possèdent, comme tout les objets, d'une propriété nommée 'prototype' qui nous sera bien utile plus tard. En ES5, **pour émuler des classes, il est donc très commun d'utiliser des fonctions !**

Notion de constructeur

Pour **émuler le comportement des classes nous utilisons des fonctions en ES5**, et il se trouve que le corps (la suite des instructions contenues dans la fonction) de ce mêmes fonctions peut très bien jouer le rôle des fonction constructrice (ou constructeur).

Ainsi, une **fonction** va pouvoir nous servir à la fois de **pseudo classe ET de constructeur** pour de nouveaux objets crée grâce à elle, ce qui est plutôt pratique ! L'objet nouvellement crée disposera d'une propriété **constructor** qui fera ainsi référence à la fonction qui a servi à le créer, les objets 'classiques', créés à la volée, ont comme constructeur la classe Object.

```
function Personnage(param_name)
{
  // on peut traiter le paramètre comme si on était dans une
  // fonction constructrice d'un 'vrai' objet de langage orienté objet
  // par classe
}

// on crée un nouvel objet Personnage
var darkvador = new Personnage("dark vador");
```

Notion de destructeur

En Javascript, contrairement au constructeur, il n'existe pas de mécanisme pouvant s'apparenter à une fonction destructrice. En effet, les objets sont supprimés à la main par le développeur, ainsi que tout les objets qui lui appartiennent en passant leur valeur à null.

En faisant cela, Javascript va indiquer au Garbage Collector (ramasse miettes) que l'objet est éligible à la destruction, il va donc pouvoir libérer l'espace occupé en mémoire. Le développeur devra donc, s'il souhaite en avoir une, coder sa propre fonction destructrice selon ses propres conventions de nommage ET l'appeler explicitement avant de passer la valeur de l'objet à null.

```
function Personnage() {  
  // constructeur  
}  
  
// exemple fonction destructrice  
Personnage.prototype.destroy = function () {  
  // libère les propriétés de l'objet  
}  
  
// on crée un objet de type Personnage  
var anakin = new Personnage();  
  
// invocation de destructeur + instance = null  
anakin.destroy();  
anakin = null;
```

Le mot clé prototype

En Javascript, toutes les données non scalaires (Array, List, Object, Function, String etc...) héritent en réalité de la classe Object et ce, même s'il n'y a pas de mécanisme d'héritage au langage à disposition du développeur.

Cela veut dire que même **une fonction est un objet**, un peu spécial certes, mais un objet quand même. En tant que tel, il **dispose donc des méthodes et propriétés** définies sur Object, notamment la **propriété prototype**.

Cette propriété contient un objet un peu spécial, car il va être utilisé par le moteur Javascript pour créer nos nouvelles instances à l'aide de nos pseudo classes. En effet, au moment de la création d'une instance, **le prototype de la fonction** qui a servi en tant que pseudo classe va être analysé par le **moteur Javascript**, et ce dernier va **copier l'ensemble des propriétés et méthodes** décrites sur la nouvelle 'instance'.

```
function Personnage(){  
  
  // création d'une propriété name sur le prototype  
  Personnage.prototype.name = "no_name";  
  
  // crée un objet de 'type' Personnage  
  var santa = new Personnage();  
  
  // retourne no_name (valeur par défaut), la propriété existe !  
  console.log(santa.name);  
}
```

Le mot clé this

Le mot clé `this` en Javascript fait référence au contexte d'exécution en cours, ou à celui qui a invoqué la fonction/méthode dans laquelle on se trouve (et qui donc devient le contexte en cours). Cela peut créer quelques problèmes parfois, selon la façon dont une méthode a été invoquée, `'this'` ne pointera pas nécessairement vers l'objet 'propriétaire' de la méthode.

```
function Personnage(p_name) {  
  this.name = p_name; // this = objet nouvellement crée  
}  
Personnage.prototype.name = "no_name";  
  
Personnage.prototype.sayHello = function () {  
  console.log(this.name);  
};  
  
var perso = new Personnage("Black Beard");  
  
// ici, grâce à la syntaxe 'à point', Javascript va déterminer que le  
// contexte de la fonction est l'objet perso, this, pointera donc  
// vers le bon objet au sein de la méthode sayHello  
perso.sayHello();  
  
// ici, on stocke une 'référence' à la méthode sayHello et on l'utilise  
// en faisant cela, on a crée une variable myfunc au sein de l'espace global  
// ce qui en fait automatiquement une de ses propriétés  
var myfunc = perso.sayHello;  
  
// ici, appeler myfunc directement ou window.myfunc revient au même,  
// this pointera donc vers window au sein de la méthode sayHello  
myfunc(); window.myfunc();
```

Héritage en Javascript

L'héritage est complètement absent en Javascript dans sa version ES5 (en ES6, il s'agit avant tout de sucre syntaxique). Cependant, il est tout à fait possible d'émuler l'héritage, il suffit pour cela de copier le prototype de la 'classe mère' sur le prototype de la 'classe fille' avant de définir les propriétés et méthodes propres à cette dernière.

Pour appeler des méthodes parentes, il suffit d'invoquer la méthode définie sur le prototype de la classe mère en précisant le contexte d'exécution comme étant celui de la 'classe fille' (à l'aide de la méthode 'apply' ou 'call').

```
function Car() {}

Car.prototype.start = function () {
  console.log("Engine started");
}

function Deloreane() {}

// copie le prototype de Car sur le prototype de Deloreane
Deloreane.prototype = Object.create(Car.prototype).prototype;

// équivalent d'une surcharge
Deloreane.prototype.start = function () {
  // équivalent de super
  Car.prototype.start.apply(this);
  console.log("... 2.21 GigaWatts !");
}
```

Interfaces en Javascript

Il n'existe pas à proprement parler d'interface en Javascript, en fait il est nécessaire encore une fois d'émuler le comportement des vraies interfaces. L'une des façons simples de faire est de créer un objet (et non pas une fonction car on pourrait l'instancier) qui contient des méthodes vides et des propriétés de valeurs nulles.

Par la suite, il faudra copier l'ensemble des propriétés et méthodes de cette interface sur le prototype de la 'classe' qui souhaite l'implémenter, ainsi elle respectera de facto le contrat passé avec l'interface.

```
var IPersonnage = {  
  sayHello: function() {},  
  name: null  
};  
  
function Personnage() {}  
  
// emulation d'une implementation d'interface IPersonnage  
for( var prop in IPersonnage ){  
  Personnage.prototype[prop] = IPersonnage[prop];  
}
```

Classes abstraites en Javascript

Il n'existe pas de classes abstraites en Javascript, car il n'existe pas de classes tout court. L'une des façons d'émuler le comportement des classes abstraites est de créer une pseudo classe (fonction) qui va servir de classe abstraite et qui va vérifier si le constructeur est celui de la classe abstraite. Si oui, on jette une erreur, sinon tout est ok.

```
//  
// création d'une pseudo classe abstraite  
function AbstractPersonnage() {  
  if (this.constructor == AbstractPersonnage)  
    throw (new Error("don't instanciate abstract class !"));  
}  
  
AbstractPersonnage.prototype.name = "no_name";  
  
function Personnage() {  
  AbstractPersonnage.apply(this); // call super constructor  
}  
  
// héritage ...  
for (var prop in AbstractPersonnage.prototype) {  
  Personnage.prototype[prop] = AbstractPersonnage.prototype[prop];  
}  
  
new Personnage(); // tout est ok  
new AbstractPersonnage(); // jette une erreur  
//
```

L'objet window

L'objet window représente la fenêtre elle-même. La propriété document d'un objet window pointe vers le document DOM chargé dans cette fenêtre. L'objet window représentant la fenêtre correspondant à un document donné peut être obtenue à l'aide de la propriété document.defaultView.

L'objet window n'est pas partagé entre les onglets faisant partie de la même fenêtre. Certaines méthodes, comme window.resizeTo et window.resizeBy s'appliquent à la fenêtre entière et non à l'onglet spécifique auquel l'objet window appartient. En général, ce qui ne peut pas raisonnablement être limité à un onglet se rapporte à la fenêtre à la place.

Lorsqu'on crée une variable dans l'espace global en Javascript, on crée en vérité une nouvelle propriété pour l'objet window.

```
// créer une variable globale revient à ajouter une propriété à l'objet window  
var myprop = "myvalue";  
console.log(window.myprop);
```

L'objet document

Toute page web chargée dans un navigateur web a son propre objet document. Cet objet sert de point d'entrée au contenu de la page et apporte des fonctions générales au document. L'interface de l'objet document permet d'accéder à de nombreuses choses comme le type de document, son URL, l'arbre DOM (incluant des éléments tels que `<body>` ou encore `<table>`), tout en fournissant des méthodes pour créer et manipuler chacun des sous-nœuds du document.

Notez que certaines APIs du DOM ne sont pas disponibles dans tous les navigateurs web, pour différentes raisons :

- Obsolète : en cours de suppression dans les navigateurs web supportant cette méthode/propriété
- Non-standard: il s'agit soit d'une fonctionnalité expérimentale pas (encore?) acceptée par tous les fabricants, ou une fonctionnalité spécifique au code du navigateur (par exemple Mozilla a quelques APIs DOM créés pour la création d'add-ons et le développement d'applications).
- Ou il s'agit d'un standard émergent ou terminé mais qui n'a pas (encore?) été implémenté par les navigateurs web.

De manière générale, tout les navigateurs modernes implémentent désormais une API DOM unifiée et supportée dans son intégralité, seuls les anciens navigateurs posent encore problème, c'est pour ça que des librairies telles que jQuery, spécialisées dans la manipulation transparente du DOM, sont encore majoritairement utilisées.

L'objet navigator

L'objet Navigator représente l'état et l'identité de l'agent utilisateur et peut être récupéré via la propriété en lecture seul `window.navigator`. De manière générale, l'objet navigator peut permettre au développeur de récupérer des informations telles que:

- Le navigateur utilisé (via la propriété `userAgent`)
- L'état de la batterie (via Battery API)
- La langue préférée de l'utilisateur
- ... Etc plein d'autres !

```
// affiche une chaîne de caractères donnant des informations
```

```
// sur le navigateur utilisé
```

```
console.log(window.navigator.userAgent);
```

```
// affiche la langue préférée de l'utilisateur
```

```
console.log(window.navigator.language);
```

```
// retourne un objet de type BatteryManager
```

```
console.log(window.navigator.battery);
```

L'objet screen

En Javascript, l'objet Screen, accessible via la propriété screen de l'objet window (window.screen) nous sert à récupérer des informations à propos de l'écran sur lequel s'affiche la fenêtre courante.

```
// affiche les hauteur/largeur de l'écran en pixels
console.log( window.screen.height );
console.log( window.screen.width );

// affiche l'orientation de l'écran
console.log( window.screen.orientation );

// bloque/débloque l'orientation de l'écran
window.screen.lockOrientation();
window.screen.unlockOrientation();
```

L'objet location

L'objet Location (accessible via `window.location`) contient des informations concernant l'URL actuelle du document et fournit des méthodes pour modifier cette URL. Cette propriété peut être utilisée pour charger une autre page.

```
// affiche l'hôte, çàd une chaîne de la forme: domaine:port
console.log(window.location.host);

//affiche les paramètres çàd une chaîne: ?param=value&param=value
console.log(window.location.search);

//affiche l'url entière
console.log(window.location.href);

//recharge la page courante
window.location.reload();

//redirige vers la page monsite.fr
window.location.assign("http://monsite.fr");

//redirige vers la page monsite.fr, n'apparaîtra pas dans l'historique
window.location.replace("http://monsite.fr");
```

L'objet History

La propriété en lecture seule `window.history` renvoie une référence à l'objet History, qui offre la possibilité de manipuler l'historique de session du navigateur (pages visitées dans l'onglet ou le cadre dans lesquels la page courante est chargée).

Pour des raisons de sécurité, l'objet History n'autorise pas l'accès au URL d'autres pages présentes dans l'historique de la session mais autorise d'y naviguer.

Il n'y a pas de possibilité de nettoyer l'historique de la session ou de désactiver les boutons précédent et suivant. La solution éventuelle la plus proche serait la méthode `location.replace()`, permettant de remplacer l'élément actuel de l'historique de la session par l'URL fournie.

```
// retourne en arrière d'un cran dans l'historique
window.history.back();

// avance d'un cran dans l'historique
window.history.forward();

// retourne en arrière de 2 crans dans l'historique
window.history.go(-2);

// retourne la longueur de l'historique
console.log( window.history.length );
```

Création d'objet au format JSON

JSON (JavaScript Object Notation) est un format d'échange de données. Bien qu'il n'en soit pas un sous-ensemble au sens strict, JSON ressemble fortement à un sous-ensemble de la syntaxe JavaScript. Le format JSON est accepté par de nombreux langages de programmation.

JSON permet de représenter des nombres, des booléens, des chaînes de caractères, la valeur null, des tableaux (séquences ordonnées de valeurs) et des objets (correspondances chaînes-valeurs) composés de ces valeurs (ou d'autres tableaux ou objets). Il ne permet pas, nativement, de représenter des données plus complexes comme des fonctions, des expressions rationnelles ou des dates.

```
//objet au format json sous forme de paires clés/valeurs
```

```
{  
  "name":"dark vador",  
  "age": 50,  
  "isJedi":false  
}
```

```
//tableau au format json
```

```
["dark vador", 80, { "mykey":"myvalue"}, true, null ]
```

ECMAScript 2015 ET AU DELÀ

Bonnes pratiques EcmaScript 5

Javascript est un langage qui évolue en permanence, ceci dit, dans sa version EcmaScript 5, les bonnes pratiques restent globalement inchangées. Les voici en versions condensées, et bien entendu cette liste n'est en rien exhaustive.

- Eviter les variables globales
- Déclarer les variables locales à l'aide du mot clé "var"
- Le mode strict doit être utilisé
- Eviter la fonction eval()
- Organiser son code en module
- Une expression doit toujours se terminer par un point-virgule
- Le code doit être découpé en plusieurs fichiers

... Et bien d'autres que vous pourrez retrouver par exemple à l'adresse suivante :
<https://maxlab.fr/javascript/bonnes-pratiques-javascript-pour-lentreprise>

ES7/ES6/ES2015, présentation générale.

ECMAScript est un ensemble de normes concernant les langages de programmation de type script et standardisées par Ecma International dans le cadre de la spécification ECMA-262. Il s'agit donc d'un standard, dont les spécifications sont mises en œuvre dans différents langages de script, comme JavaScript ou ActionScript, ainsi qu'en C++ (norme 2011). C'est un langage de programmation orienté prototype.

Pour le langage Javascript, la norme ECMAScript 5 est aujourd'hui la plus répandue, toutefois certains navigateurs, comme Chrome (Google Inc.), mettent déjà à disposition des développeurs une implémentation de Javascript respectant la norme ECMA 6 (ES2015).

Nommée ES2015, la dernière version d'ECMAScript a été publiée en juin 2015⁶. Son support par les navigateurs évolue progressivement, mais il est possible d'utiliser un transcompilateur (tel que Babel.js⁸) vers ES5 pour développer dès aujourd'hui en ES6.

Première version publiée après le changement du processus de normalisation, l'ES7, ou ES2016, apporte peu de changements au langage. Le nouveau processus prévoit en effet de publier chaque année une nouvelle norme avec les ajouts qui ont eu lieu dans l'intervalle, afin d'éviter de publier des changements énormes comme ce fut le cas en 2015⁹.

ECMAScript 6 et 7, les nouveautés

Les normes ECMAScript 6 et 7 arrivent avec leurs lots de nouvelles fonctionnalités notamment les suivants (mais pas que):

- Les mot-clés let et const
- Les templates et chaînes de caractères
- Les paramètres par défaut
- Les promesses
- Les fonctions lambda (arrow functions)
- Les classes et l'héritage

Nous allons détailler un peu plus chacune de ces fonctionnalités.

Les mot-clés let et const

Le mot clé **let** permet de déclarer une variable limitée à la portée d'un bloc, c'est-à-dire qu'elle ne peut être utilisée que dans le bloc où elle a été déclarée, ce qui n'est pas le cas avec var.

Le mot-clé **const** sert à définir une valeur immuable, accessible uniquement en lecture seule.

```
function wholsTheSithLord() {  
  // la valeur ne peut plus être modifiée  
  const MOVIE = "STAR WARS";  
  
  if (true) {  
    // existe en dehors du bloc conditionnel  
    var sith_lord = "Palpatine";  
    // existe uniquement dans le bloc conditionnel  
    let padawan = "Dark Vador";  
  }  
  
  console.log(sith_lord); // displays Palpatine  
  console.log(padawan); // Uncaught Reference Error  
  console.log(MOVIE);  
}  
  
wholsTheSithLord();
```

Templates Strings

Les "templates strings" permettent d'insérer plus facilement des valeurs de variables au sein d'une chaîne de caractère, ces dernières peuvent également être définies sur plusieurs lignes.

```
let jedi = {surname:"Obiwan", name:"Kenobi"};  
let msg = `${jedi.name} ${jedi.surname} is the Jedi Master`;  
console.log(msg);
```

Paramètres par défaut

Il est dorénavant possible de donner une valeur par défaut aux paramètres, cela évite les lignes de codes superflues et facilite la lecture du code.

```
function wholsTheSithLord(p_name='Dark Vador')
{
  console.log("The Sith lord is: ", p_name);
}

wholsTheSithLord();
```

Les fonctions fléchées/lambda

Il s'agit d'une nouvelle façon d'écrire les fonctions anonymes, plus compacte et permettant de lier définitivement le contexte d'exécution de la fonction (this).

```
function launchFunc(p_function) {
  p_function();
}

// ecma 5 style anonymous function
launchFunc(
  function () {
    console.log("I am The Sith Lord")
  }
);

// ecma6 arrow function
launchFunc(
  () => { console.log("I am The Sith Lord") }
);
```

Les promesses

L'objet Promise (pour « promesse ») est utilisé pour réaliser des traitements de façon asynchrone. Une promesse représente une valeur qui peut être disponible maintenant, dans le futur voire jamais.

L'interface Promise représente un intermédiaire (proxy) vers une valeur qui n'est pas nécessairement connue au moment de sa création. Cela permet d'associer des gestionnaires au succès éventuel d'une action asynchrone et à la raison d'une erreur. Ainsi, des méthodes asynchrones renvoient des valeurs comme les méthodes synchrones, la seule différence est que la valeur retournée par la méthode asynchrone est une promesse (d'avoir une valeur plus tard).

```
new Promise((resolve, reject) => {
  const xhr = new XMLHttpRequest();
  xhr.open("GET", url);
  xhr.onload = () => resolve(xhr.responseText);
  xhr.onerror = () => reject(xhr.statusText);
});

function resolve(p_msg){
  console.log("resolve: ", p_msg);
}

function reject(p_msg){
  console.log("reject: ", p_msg);
}
```

Classes et Héritages

Ecma 6 introduit des mécanismes Orienté Objet par classe grâce aux mots-clés **class** et **extends**, malheureusement ces derniers n'introduisent aucun changement dans la manière dont les objets sont réellement agencés et conçus au sein de la VM Javascript, il s'agit uniquement de sucre syntaxique.

```
class Personnage {
  constructor() {
    this.name = "";
  }

  sayMyName() {
    console.log(this.name);
  }
}

class JediKnight extends Personnage {
  constructor(p_name) {
    this.power = 250;
    this.name = p_name;
  }
}

let obiwan = new JediKnight("Obiwan Kenobi");
obiwan.sayMyName();
```

Les Observables

Un Observable est un producteur de données qui peut être observé. On le mettra sous observation avec la méthode **subscribe** et cette observation sera exécutée par un objet de type Observer. Les Observables se sont imposés dans le monde de la programmation réactive (reactive programming) par le biais de la librairie Reactive X, abrégé RxJs pour sa version Javascript. Un Observable peut également produire des données de façon asynchrone et il est possible de fusionner plusieurs Observables entre eux.

Il faut bien noter que les Observables sont souvent associés aux promesses, voire décrits comme des super-promesses, ce qui n'est pas tout à fait exact, un article entier a d'ailleurs été rédigé à l'attention des développeurs à l'adresse suivante: <https://medium.com/@benlesh/learning-observable-by-building-observable-d5da57405d87> et explique un peu plus en détail ce que sont les Observables.

```
Rx.Observable.create(function(observer) {  
  setTimeout(() => observer.next("valeur A"), 700);  
  setTimeout(() => observer.next("valeur B"), 400);  
})  
.subscribe(e => console.log(e));  
  
// affiche "valeur A", puis "valeur B" dans la console
```

Reactive Extension For Javascript (RxJS)

Reactive Extensions (Rx) est une bibliothèque permettant de développer des applications "orientées événements" et utilisant des requêtes asynchrones en liant entre elles des séquences d'objets de type Observables.

Les séquences de données peuvent prendre plusieurs formes, telle qu'un flux de données issue d'un webservice, des requêtes vers des webservices, des notifications systèmes, ou bien encore une séquence d'événements utilisateurs etc...

Rx représente toutes ces données sous la forme d'une séquence d'Observables, une application peut souscrire à ce flux de données et être notifiée chaque fois qu'une donnée est ajoutée au flux.

La comparaison avec les promesses est alors tentante et il est possible de se faire une idée plus précise de ce que sont les Observables et dans quels domaines les préférer par rapport aux Promises à l'adresse suivante: <https://xgrommx.github.io/rx-book/index.html>

Les outils

L'une des choses les plus difficiles à déterminer lorsque l'on commence un développement en Angular est la stack complète de développement. Fort heureusement, une multitude d'outils tous plus performants les uns que les autres sont venus renforcer l'écosystème Javascript ces derniers temps, et ce, grâce à **node.js** et son gestionnaire de dépendances spécialisé front-end **npm**. L'on peut donc compter au rang des outils quasiment indispensables:

- Node.js et NPM (node package manager)
- Typescript (langage poussé par Google pour le développement Angular)
- Un bon éditeur Typescript (Microsoft Visual Code)
- Un outil de scaffolding (génération de structure de projets et codes) comme `@angular/cli`
- Un module loader, comme Webpack
- Un bon navigateur et/ou environnement d'exécution (Google Chrome, Ionic ...)

Le langage Typescript

TypeScript est un langage de programmation libre et open source développé par Microsoft qui a pour but d'améliorer et de sécuriser la production de code JavaScript. C'est un sur-ensemble de JavaScript (c'est-à-dire que tout code JavaScript correct peut être utilisé avec TypeScript). Le code TypeScript est transcompilé en JavaScript, pouvant ainsi être interprété par n'importe quel navigateur web ou moteur JavaScript.

TypeScript permet un typage statique optionnel des variables et des fonctions, la création de classes et d'interfaces, l'import de modules, tout en conservant l'approche non-contrainante de JavaScript. Il supporte la spécification ECMAScript 6.

- Typage statique
- Typage générique
- Interfaces
- Classe, classe abstraite, expressions de classe
- Modules
- Mixin
- Enumérations
- Paramètres optionnels
- Symboles

... Et bien d'autres, d'ailleurs depuis la version 1.6, le format JSX est supporté.

ES6 Module Pattern

La norme ES6 prévoit de fournir au développeur la possibilité de regrouper et emballer des objets, classes, fonctionnalités et valeurs au sein de **modules**, dans la continuité des pattern modulaires déjà bien ancrés au sein de l'écosystème Javascript (AMD par exemple).

L'instruction `export` est utilisée pour permettre d'exporter des fonctions et objets ou des valeurs primitives à partir d'un fichier (ou module) donné. Ces fonctions et objets peuvent ensuite être utilisés dans d'autres fichiers grâce à `import`.

Cette fonctionnalité n'est pas encore implémentée nativement par la plupart des navigateurs, en revanche, elle est gérée par de nombreux transpileurs comme **Traceur**, **Babel** ou **Rollup**.

```
function wholsTheSithLord() {  
  console.log("Palpatine is the Sith Lord !");  
}  
  
// on exporte une fonction déclarée plus haut  
export { wholsTheSithLord };  
  
// on exporte une constante  
export const DARK_VADOR = "Dark Vador";
```

ES7 Async functions

La déclaration `async function` définit une fonction asynchrone qui renvoie un objet `AsyncFunction`. On peut également définir des fonctions asynchrones grâce au constructeur `AsyncFunction` et via une expression de fonction asynchrone.

Lorsqu'une fonction asynchrone est appelée, elle renvoie une promesse. Lorsque la fonction asynchrone renvoie une valeur, la promesse est résolue avec la valeur renvoyée. Lorsque la fonction asynchrone lève une exception, la promesse est rompue avec la valeur de l'exception.

Une fonction asynchrone peut contenir une expression `await` qui permet d'interrompre l'exécution de la fonction asynchrone en attendant la résolution d'une promesse passée à l'expression. L'exécution de la fonction asynchrone reprend lorsque la promesse est résolue.

L'objectif des fonctions asynchrones avec `await` est de simplifier le comportement des promesses lors d'opérations synchrones et d'opérer sur des groupes de promesses. Si les promesses sont en quelque sorte des callbacks organisés, `async/await` permet de combiner les générateurs et les promesses. On peut trouver des exemples à l'adresse suivante: https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Instructions/async_function

Module Loader & Webpack

Un module loader est un outil qui fournit une solution permettant de gérer des modules Javascript, en respectant la norme des ES6 Modules. Il s'agit en règle générale d'une suite logicielle composée d'un utilitaire en ligne de commande et de fichiers à configurer.

L'un des plus célèbres Module Loader de l'écosystème Javascript est Webpack, ce dernier permet, comme son nom l'indique, de créer des paquetages à partir de fichier Javascript (mais pas que) et de les mettre à disposition sous forme de modules au sein d'une application front-end.

Pour plus de renseignements sur le fonctionnement de Webpack, rendez-vous sur le site officiel du produit à l'adresse suivante: <https://webpack.github.io/docs/>

LA PROGRAMMATION EVENEMENTIELLE

Javascript, un langage évènementiel

Javascript fait partie de la famille des langages évènementiels comme Actionscript, Lingo, Qt en C++ (bibliothèque), cela signifie qu'il nous procure, de manière native (ou à l'aide de bibliothèques comme jQuery par exemple), des mécanismes pour programmer de façon évènementielle.

En informatique, la programmation évènementielle s'oppose à la programmation séquentielle. Le programme sera principalement défini par ses réactions aux différents événements qui peuvent se produire, çàd des changements d'état de variable, l'incréméntation d'une liste, un mouvement de souris ou de clavier...

En Javascript, elle est surtout utilisée avec les objets du DOM, mais d'autres objets peuvent être utilisés en programmation évènementielle. Pour cela, il faut utiliser conjointement les méthodes suivantes des objets implémentant l'interface `EventListener`:

- **addEventListener**: Qui nous permet d'ajouter un écouteur d'événements (une fonction de rappel) qui se déclenchera en réaction à un événement.
- **removeEventListener**: Qui nous permet d'enlever un écouteur d'événements ajouté auparavant.
- **dispatchEvent**: Qui nous permet de lancer un événement à la main, s'utilise avec les objets de type Event
- **Objet Event**: Cet objet est utilisé pour créer des événements déjà connus ou personnalisés, attention les événements utilisateurs tels que les clicks de souris, les frappes au clavier ne peuvent pas être simulés parfois, pour des raisons de sécurité.

Les évènements de chargement de page

En Javascript, il est courant d'attendre que la page soit chargée avant de déclencher un script quelconque. Pour cela, il existe plusieurs méthodes, notamment une qui nous dicte d'inclure le script en tant que dernier enfant du noeud body.

Parfois, cela peut être une bonne méthode, mais le plus souvent, il s'agit d'une mauvaise pratique. En effet, il existe deux évènements spécifiques au chargement d'une page, l'un envoyé par l'objet window, et l'autre par l'objet document:

- **load event:** Envoyé par l'objet window lorsque l'ensemble des noeuds HTML de la page et toutes les ressources qui y sont associées (images, vidéos, sons etc...) sont parsés, téléchargés, prêts à être manipulés.
- **DOMContentLoaded event:** Envoyé par l'objet window.document lorsque l'ensemble du HTML de la page est parsé et prêt à être manipulé. On ne tient pas compte des autres ressources.

```
function onPageFullyLoaded(event){
  console.log("HTML & other ressources fully loaded");
}

function onHTMLFullyLoaded(event){
  console.log("HTML fully loaded");
}

window.addEventListener("load", onPageFullyLoaded, false );
window.document.addEventListener("DOMContentLoaded", onHTMLFullyLoaded, false);
```

Les évènements utilisateur

Il est possible de détecter des évènements déclenchés par une action de l'utilisateur comme par exemple:

- Le click souris: à l'aide de l'évènement click'
- Passer par dessus un élément: à l'aide de l'évènement mouseover
- Appuyer sur une touche du clavier: à l'aide de l'évènement keydown

```
function keyDownHandler(event){
  console.log("appui sur une touche");
}
window.addEventListener("keydown", keyDownHandler, true);

function clickHandler(event){
  console.log("click souris");
}

window.addEventListener("click", clickHandler, true);
```

Il en existe beaucoup d'autres que vous pouvez découvrir sur la documentation officielle Javascript de Mozilla Developer Network: <https://developer.mozilla.org/fr/>

Différences entre les navigateurs (non modernes)

Il existe des différences d'API entre les navigateurs les plus anciens en ce qui concerne la programmation événementielle. Par exemple, au lieu de la méthode standardisée **addEventListener** le navigateur Internet Explorer a très longtemps utilisé la méthode **attachEvent** ainsi que les méthodes **detachEvent** et **fireEvent**.

Mais il n'y a pas qu'Internet Explorer qui pouvait poser problème, à l'époque chaque navigateur avait ses propres subtilités et il était assez compliqué de programmer efficacement de façon événementielle. C'est en partie pour répondre à ce besoin qu'a été créée la bibliothèque **jQuery**.

COMMUNICATION EN TEMPS REEL

Problématiques et définition

La communication en temps réel permet à deux process d'un même programme (ou de programmes différents) de communiquer par le biais de messages, et ce, sans faire appel à des opérations bloquantes. Les messages sont envoyés, directement au destinataire ou à un programme tiers qui fait alors office de gestionnaire de message.

Les messages sont alors consommés par le destinataire, et celui-ci peut choisir d'y répondre ou non sans limite de délai. Bien entendu une telle façon de communiquer doit être accompagnée d'une façon de programmer bien précise. Le javascript, de part sa nature asynchrone, répond très à ce défi, notamment grâce à sa gestion de l'asynchronicité et à son modèle événementiel.

Il existe plusieurs technologies, natives ou non, nous permettant de mettre en place une communication en temps réel inter processus en javascript. Redis, RabbitMQ, Socket.IO ou encore les websockets seules sont autant de solutions nous permettant de répondre à nos besoins en la matière.

Websockets et Socket.IO

Socket.IO est une bibliothèque JavaScript pour les applications Web en temps réel. Il permet une communication bidirectionnelle en temps réel entre les clients Web et les serveurs. Il comporte deux parties: un côté client bibliothèque qui fonctionne dans le navigateur , et un côté serveur bibliothèque pour Node.js . Les deux composants ont une API presque identique . Comme Node.js , il est piloté par les événements.

Socket.IO utilise principalement le protocole WebSocket avec l'interrogation comme option de secours, tout en fournissant la même interface. Bien qu'il puisse être utilisé comme un simple wrapper pour WebSocket, il fournit de nombreuses autres fonctionnalités, notamment la diffusion vers plusieurs sockets, le stockage des données associées à chaque client et les E / S asynchrones. Il peut être installé avec l'outil npm.

socketio client

```
const io = require("socket.io-client");
const socket = io("ws://localhost:3000");

function start(){
  socket.send("classic client")
  socket.emit("clientEvent", "custom client");
}

socket.on("connect", start);
socket.on("message", console.log);
socket.on("serverEvent", console.log);
```

socketio server

```
const io = require("socket.io")(3000);

function init(socket){
  socket.on("message", console.log );
  socket.send("classic server");

  socket.on("clientEvent", console.log );
  socket.emit("serverEvent", "custom server");
}

io.on("connection", init);
```

TP: Server Side Events & Websockets

Le but de ce TP est de créer un tchat en temps réel à l'aide de Websockets. Le tchat devra comporter les fonctionnalités suivantes:

- Il doit y avoir un salon principal, où tout le monde peut s'exprimer.
- Il doit y avoir la possibilité de créer un salon privé entre deux utilisateurs.
- Il doit y avoir la possibilité de fermer un salon privé entre deux utilisateurs.
- BONUS: Il doit y avoir la possibilité d'envoyer des emojis

L'utilisation de socket.io est recommandée quoique non obligatoire. La durée maximale du TP est de 1 heure.

NOTIONS AVANCEES

Les valeurs truthy/falsy

Si une valeur peut être convertie en true, on dit en anglais qu'elle est truthy. Pour false on dit qu'elle est falsy. Parmi les expressions qui peuvent être converties en false, citons celles qui seront évaluées à :

- null
- 0
- la chaîne vide ("")
- undefined.

Même si les opérateurs && et || peuvent être utilisés avec des opérandes qui ne sont pas des valeurs booléennes, ils peuvent toujours être considérés comme des opérateurs booléens puisque leurs valeurs de retour peuvent toujours être converties en valeurs booléennes.

Comme les expressions logiques sont évaluées de gauche à droite, leur évaluation sera éventuellement « court-circuitée » à l'aide des règles suivantes :

l'évaluation de false &&n'importe quoi est court-circuitée en false.

l'évaluation de true ||n'importe quoi est court-circuitée en true.

Les règles de la logique garantissent que ces évaluations seront toujours correctes. Notons que la partie n'importe quoi des expressions mentionnées ci-dessus ne sera jamais évaluée, et que tout effet de bord éventuel induit par cette évaluation ne se produira pas.

L'opérateur in

L'opérateur in renvoie true si la propriété donnée appartient à l'objet donné et s'utilise avec la syntaxe suivante: **ma_propriete in nomObjet**. Ici, 'ma_propriete' est une expression évaluée en un nombre ou une chaîne de caractères qui représente le nom d'une propriété ou l'indice d'un tableau et 'nomObjet' représente l'objet que l'on souhaite inspecter.

```
var tab = ["New York"];

// affiche true
console.log(0 in tab);

// affiche false car il n'existe pas d'éléments à l'indice 1 dans le tableau
console.log(1 in tab);

// affiche true, length est une propriété des objets de type tableau
console.log("length" in tab)

// on peut utiliser l'opérateur in dans une boucle for .. in
var obj = { "age": 40, "name": "Dark Vador" }, prop = null;
for (prop in obj) {

    // affiche tour à tour "age" et "name"
    console.log(prop);
}
```

L'opérateur delete

L'opérateur delete permet de retirer une propriété donnée d'un objet. Lorsque la suppression se déroule sans problème, l'opération renvoie true, sinon c'est la valeur false qui est renvoyée. Voici quelques scénarios importants qui précisent ce comportement :

- Si une propriété du même nom existe sur la chaîne de prototypes ne peuvent être supprimées
- Toute propriété déclarée avec var ne peut pas être supprimée de la portée globale ou de la portée d'une fonction.
- Les propriétés propriétés des objets natifs comme Array ne peuvent être supprimées.

```
var obj = {nom: "Dark Vador"};

// la propriété est supprimée, renvoie true
delete obj.nom;

// renvoie false et lève une erreur
delete Array.splice;
```

Conversions de valeurs

Il existe 2 façons de convertir des valeurs en Javascript, la première consiste à convertir soi-même les données, la seconde consiste à laisser Javascript opérer ce que l'on appelle une **conversion implicite**.

Pour les conversions explicites, faites à la main, il existe un panel de fonctions telles que `parseInt`, `parseFloat`, les types de données eux-mêmes (`String`) (`Number`) (`Date`) etc ...

Mais il existe des cas où Javascript va chercher à convertir une donnée dans un autre type de donnée, essentiellement lors des comparaisons non strictes de données. Dans cette situation **La donnée du type le plus complexe va être convertie dans le type de données le plus simple**.

```
var age = "30.5";

// convertit la chaîne en nombre flottant
console.log(parseFloat(age));

// convertit la chaîne en nombre entier
console.log(parseInt(age));

// conversion implicite de données, le type le plus complexe (String)
// va être automatiquement converti en (Number) par Javascript
console.log( age == 30.5 );
```

Egalité stricte

Il existe 2 mécanismes de comparaisons en Javascript, la comparaison stricte et la comparaison non stricte. La comparaison non stricte est la plus employée car elle utilise l'opérateur `==`. Cet opérateur va tenter de comparer les valeurs contenues dans les 2 opérandes mais ne va pas se soucier du type de données desdits opérandes.

La comparaison stricte utilise l'opérateur `===`, cet opérateur va comparer les valeurs des 2 opérandes ainsi que les types de données. Ainsi, aucune opération de conversion implicite ne sera effectuée lors de la comparaison.

```
// renvoie true, seules les valeurs sont comparées.  
console.log("10" == 10);  
  
// renvoie false, valeurs & types comparés  
console.log("10" === 10);
```

Programmation fonctionnelle

La programmation fonctionnelle est une approche de la programmation différente de l'approche orientée objet, à laquelle elle peut accessoirement se combiner. Elle produit des abstractions de langage utiles en combinant astucieusement des fonctions, de sorte à ce que le code soit plus facile à lire.

Par exemple, l'itération au sein d'une liste peut être décrite comme la combinaison de 2 actions, l'une nous permet de boucler sur la liste, la seconde nous permet d'effectuer une action sur l'élément en cours. Dans l'exemple ci-dessous, on réduit un tableau à la somme de tout ces éléments:

```
function reduce(action, tab, base) {  
  for (var i = 0; i < tab.length; i++) {  
    base = action(base, tab[i]);  
  }  
  return base;  
}  
  
function add(a, b){  
  return a + b;  
}  
  
// nous retourne la somme des éléments du tableau  
console.log( reduce(add, [0,30,80,100], 0 ));
```

Masquage et portée des variables

En Javascript, une variable est accessible dans l'environnement dans lequel elle a été définie, çàd que si la variable est définie dans l'espace global (window), alors elle sera accessible dans tout les environnements enfants de window telles que des fonctions etc ...

En fait, à chaque fois qu'une variable est déclarée dans un environnement (par exemple une fonction), elle devient automatiquement une propriété de cet environnement, puisque tout ou presque est objet en Javascript.

```
// cette variable devient automatiquement une propriété de l'environnement
// dans lequel elle est définie (window) et est donc disponible dans tout
// les environnements "enfants"
var sith_lord = "Dark Vador";

function wholsTheSithLord(){
  // affiche Dark Vador dans la console
  console.log(sith_lord);
}

wholsTheSithLord();
```

Les closures

Une fermeture, ou closure en anglais, est une fonction qui fait utiliser des variables indépendantes (utilisées localement mais définies dans la portée englobante). Autrement dit, ces fonctions se « souviennent » de l'environnement dans lequel elles ont été créées (on dit aussi que la fonction capture son « environnement »).

Normalement, les variables locales d'une fonction n'existent que pendant l'exécution d'une fonction cependant, en Javascript, une fonction se souvient toujours de son contexte interne, çàd des variables et autres fonctions créées en son sein, elles fonctionnent donc comme des objets (car ce sont des objets).

```
function getSithLordFunc() {  
    var sith_lord = "Dark Vador";  
    function wholsTheSithLord() {  
        console.log(sith_lord);  
    }  
  
    return wholsTheSithLord;  
}  
// retourne la fonction wholsTheSithLord...  
var func = getSithLordFunc();  
  
// invoque la fonction wholsTheSithLord qui se souvient  
// de la variable sith_lord décrite dans le même contexte  
func();
```

Comprendre le hoisting (hissage)

En JavaScript, fonctions et variables sont hissées ('hoisting' est traduit en français par 'hissage'). Le hissing est un comportement du Javascript qui déplace une déclaration au début du scope dans lequel il est déclaré (scope courant de la fonction ou dans le global scope). Cela signifie qu'il est possible d'utiliser une fonction ou une variable avant qu'elle ne soit déclarée. Autrement dit, une fonction ou variable peut-être déclarée après son utilisation.

```
// affiche Dark Vador dans la console
console.log(sith_lord);

var sith_lord = "Dark Vador";

// affiche Dark Vador dans la console
wholsTheSithLord();

function wholsTheSithLord() {
  console.log("Dark Vador");
}
```

Le Module Pattern (AMD)

En Javascript, il existe un design pattern nommée 'module pattern' (abrégé amd), qui permet d'empaqueter des objets, fonctions et variables à l'intérieur d'un objet sobrement nommée 'module' créée à l'aide d'une fonction qui s'invoque elle-même et qui retourne un résultat, un objet en règle générale. Le développeur va donc pouvoir exporter, via les propriétés de l'objet retourné, les méthodes et propriétés qu'il souhaite exposer à l'extérieur du module et garder 'privées' toutes les autres.

```
var module = (  
  function () {  
  
    function _privateMethod() {  
      console.log("private method");  
    }  
  
    function publicMethod() {  
      _privateMethod();  
      console.log("public method");  
    }  
  
    return {  
      myPublicMethod: publicMethod  
    }  
  }  
)();  
  
module.myPublicMethod();
```

Apply & call

La méthode `apply()` des objets de type fonction, invoque la fonction en lui passant une valeur `this` et des arguments sous forme d'un tableau (ou d'un objet semblable à un tableau). Ainsi le développeur peut s'assurer du contexte dans lequel est invoquée la fonction lors de l'appel à celle-ci.

La méthode `call` est similaire à `apply()`, la différence fondamentale réside dans le fait que `call()` accepte une liste d'arguments, tandis que la méthode `apply()` accepte un unique tableau d'arguments.

```
function wholsTheSithLord() {  
  console.log(this.name);  
}  
  
var sith_lord = {  
  name: "Dark Vador"  
};  
  
// invoque la fonction, l'opérateur this pointera vers l'objet sith_lord  
wholsTheSithLord.apply(sith_lord);
```

Garantir le contexte (binding)

La fonction `bind()` crée une nouvelle fonction qui, lorsqu'elle est appelée, a pour contexte `this` la valeur passée en paramètre et éventuellement une suite d'arguments qui précéderont ceux fournis à l'appel de la fonction créée. Cela permet au développeur de garantir le contexte d'exécution d'une fonction et de résoudre ce type de problème:

```
var sith_lord = {
  name: "Dark Vador",
  wholsTheSithLord: function () {
    console.log(this.name);
  }
};

// affiche Dark Vador
sith_lord.wholsTheSithLord();

// affiche undefined, this vaut window ici...
var func = sith_lord.wholsTheSithLord;
func();

// affiche Dark Vador car le contexte (this) est lié à sith_lord
func = sith_lord.wholsTheSithLord.bind(sith_lord);
func();
```

La console de debug

Il existe une multitude d'outils pour nous aider à débbugger en Javascript mais le plus complet (et donc le plus utile) est dans aucun doute la console du navigateur. En effet celle-ci nous permet de poser des points d'arrêts (breakpoints), des watchers (watch), de vérifier ce qui se passe sur le réseau (panneau network) etc etc ... Voici une liste des panneaux utiles que vous pourrez trouver dans la console de Google Chrome par exemple:

- Le panneau sources
- Le panneau ressources
- Le panneau network
- Le panneau timeline
- Le panneau profiles
- Le panneau audits

Web Workers, Service Workers

Les Web Workers fournissent un moyen simple d'exécuter des scripts dans des threads en arrière-plan d'un contenu web. Une fois créé, un worker peut envoyer des messages à son thread parent. Un worker dédié est seulement accessible à partir du script qui l'a créé, tandis que les workers partagés peuvent être interrogés par de multiples scripts.

Les workers s'exécutent dans un un contexte global différent du contexte de la fenêtre courante, utiliser window pour obtenir le contexte global courant (au lieu de self) au sein d'un Worker retournera une erreur.

```
//script principal
var sith_lord = new Worker("dark_vador.js");
sith_lord.onmessage = function (event) {
  // affiche Dark Vador
  console.log(event.data);
};

sith_lord.postMessage("getsithlord");

// dark_vador.js
onmessage = function (event) {

  if (event.data == "getsithlord")
    postMessage("Dark Vador");
}
```

Les tests unitaires

Les tests unitaires sont une depuis bien longtemps, une norme dans le monde du développement Backend, toutefois on peut dire qu'au sein de l'écosystème Javascript, l'écriture et l'utilisation de tels tests n'est pas entré tout de suite dans les moeurs.

Il existe cependant depuis plus de 10 ans, des frameworks de tests unitaires disponibles pour ce langage et aujourd'hui certains rivalisent d'inventivité afin de procurer au développeur toujours plus de flexibilité et d'efficacité dans sa démarche de test.

Voici une petite liste (non exhaustive) des différents frameworks de tests unitaires les plus utilisés au moment ou ces lignes sont écrites:

- Jasmine: <https://jasmine.github.io/>
- Mocha: <https://mochajs.org/>
- Karma: <https://karma-runner.github.io/1.0/index.html>
- UnitJS: <http://unitjs.com/>

ES6

ECMAScript 2015 (sixième édition) est la version actuelle pour le langage ECMAScript. Anciennement appelé « ES6 », c'est le standard qui définit l'implémentation de JavaScript, notamment pour SpiderMonkey, le moteur utilisé par Firefox et les autres applications Mozilla.

JavaScript propose donc une version récente du langage qui remplit les spécifications de cette norme, qui apporte son lot de nouveautés notamment:

- Des classes
- Un système d'héritage
- Un système de module intégré
- Le support des lambdas fonctions
- Etc...

Utiliser ES6 aujourd'hui

ES6 est une norme récente, par conséquent, il n'est pas possible de l'utiliser avec tout les navigateurs. En effet, les moins récents ne proposent pas de support pour cette version du langage Javascript. Afin de pouvoir l'utiliser partout, il nous faudra passer notre code ES6 à un outil que l'on appelle un 'transpiler', tel que Babel. Cet outil aura la charge de convertir notre code ES6 en sa version ES5.

Pour plus de détails sur le fonctionnement de Babel, vous pouvez vous rendre sur le site <https://babeljs.io/> et commencer dès maintenant à coder en ES6. S'y mettre de suite reste une bonne idée, car le standard est maintenant finalisé et ne bougera probablement plus à l'avenir (ou très peu), une fois que tout les navigateurs le supporteront, ES5 disparaîtra et vous devrez quoiqu'il arrive vous mettre à l'ES6, autant prendre de l'avance.

INTRODUCTION À TYPESCRIPT

C'est quoi Typescript ?



TypeScript est un **langage de programmation libre et open source** développé par Microsoft qui a pour but **d'améliorer et de sécuriser** la production de code JavaScript. C'est un **sur-ensemble** de JavaScript (c'est-à-dire que tout code JavaScript correct peut être utilisé avec TypeScript). Le code TypeScript est transcompilé en JavaScript, pouvant ainsi être interprété par n'importe quel navigateur web ou moteur JavaScript.

TypeScript permet un typage statique optionnel des variables et des fonctions, la création de classes et d'interfaces, l'import de modules, tout en conservant l'approche non-contrainante de JavaScript. Il supporte la spécification ECMAScript 6.

Installer Typescript

Typescript n'est pas un langage directement disponible sur le système, il n'est pas non plus interprété directement au sein du navigateur, comme Javascript. Pour l'utiliser, nous avons besoin de l'installer sur la machine.

Il existe deux façons d'installer Typescript:

- En installant **Visual Studio** et le plugin Typescript
- En passant par le gestionnaire de paquets de node.js, **npm**

```
#permet d'installer typescript au sein du dossier courant  
npm install typescript
```

```
#ou
```

```
#permet d'installer typescript de façon globale sur la machine  
npm install -g typescript
```

```
#puis taper la commande suivante pour obtenir le numéro de version  
tsc -v
```

Un Hello World en Typescript

Voici un programme de base écrit en typescript, un 'hello world'. La syntaxe ressemble énormément à Javascript mais vous pouvez noter que l'on utilise un type de données précis pour notre variable, le type 'string' qui correspond à une chaîne de caractères.

```
// on déclare une variable str contenant le chaîne "hello world"  
let str:string = "hello world";  
  
// puis on l'affiche  
console.log(str);
```

Notez bien que l'on ne peut se servir directement de ce fichier source, en effet, aucun navigateur ou programme connu n'interprète le typescript de façon 'native'. Il faut donc passer par une étape supplémentaire nommée **transpiling** en anglais. Ce terme peut être traduit par **transcompilation**.

Transpiling

Le Typescript n'est pas (encore) un langage destiné à être directement interprété par le navigateur. Son but est d'offrir une **extension** à Javascript, un **sur-ensemble de fonctionnalités**

Il faut donc passer par une étape supplémentaire nommée **transpiling** en anglais. Ce terme peut être traduit par **transcompilation**.

```
#ici on "transcompile" le fichier "monfichier.ts" et l'on  
#obtient ainsi un fichier équivalent, traduit en javascript
```

```
tsc monfichier.ts
```

Le fichier tsconfig.json

Le fichier tsconfig.json est le cœur de la configuration TypeScript. Il permet de définir comment le compilateur tsc doit transformer le code TypeScript en JavaScript.

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "strict": true,
    "outDir": "./dist",
    "rootDir": "./src"
  }
}
```

Les options du compilateur

Dans `compilerOptions`, on trouve des paramètres importants :

- **target**: Version JS de sortie (ES5, ES6, ES2020, etc.)
- **module**: Système de modules (`commonjs`, `esnext`, etc.)
- **strict**: Active toutes les vérifications strictes (true recommandé)
- **outDir**: Dossier de sortie des fichiers compilés
- **rootDir**: Dossier racine des sources TypeScript
- **sourceMap**: Génère un `.map` pour faciliter le débogage dans le navigateur
- **noImplicitAny**: Interdit les variables sans type explicite
- **esModuleInterop**: Améliore la compatibilité avec les modules CommonJS
- **skipLibCheck**: Ignore les erreurs dans les fichiers `.d.ts`
- **include**: Fichiers ou dossiers à compiler
- **exclude**: Fichiers ou dossiers ignorés
- **files (opt)**: Si défini, alors seuls les fichiers mentionnés sont compilés

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,
    "outDir": "./dist",
    "rootDir": "./src",
    "sourceMap": true,
    "esModuleInterop": true,
    "include": ["src/**/*.ts"],
    "exclude": ["node_modules", "dist"]
  }
}
```

Créer un build

Une fois tsconfig.json configuré :

```
# Compiler le projet  
npx tsc  
  
# Compiler en watch mode  
npx tsc --watch
```

Et pour se simplifier la vie, on peut ajouter les commandes suivantes au fichier package.json :

```
{  
  "scripts": {  
    "build": "tsc",  
    "build:watch": "tsc --watch"  
  }  
}
```

Stratégies de projets à configuration multiples

Pour les gros projets ou les monorepos, il est fréquent de découper en plusieurs tsconfig. On regroupe ainsi toutes les options communes dans un fichier **tsconfig.base.json** et les options spécifiques à chaque sous projet sont contenues dans des fichiers spécifiques à chaque sous projet:

```
/monorepo
├── tsconfig.base.json
├── packages/
│   ├── frontend/tsconfig.json
│   └── backend/tsconfig.json
```

tsconfig.base.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true
  }
}
```

packages/frontend/tsconfig.base.json

```
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "outDir": "./dist"
  },
  "include": ["src"]
}
```

Création d'un workflow personnalisé

On peut créer un flux de travail complet pour TypeScript avec compilation, nettoyage et exécution.

```
{
  "scripts": {
    "clean": "rm -rf dist",
    "build": "npm run clean && tsc",
    "start": "node dist/index.js",
    "dev": "tsc --watch & nodemon dist/index.js"
  }
}
```

- **clean** : supprime les fichiers compilés
- **build** : nettoie puis compile
- **start** : lance le programme compilé
- **dev** : compile en continu et redémarre automatiquement

TypeScript Basics & Basic Types

Les types de données de bases en Typescript

- **Boolean**: Une simple valeur pouvant être vraie (true) ou fausse
- **Number**: Un nombre flottant (ex: 1.5)
- **String**: Une chaîne de caractères (ex: "Hello World")
- **Any**: Un type désignant "n'importe quel type de données", équivalent du type object en javascript.
- **Void**: Un type désignant du vide, rien, aucune valeur, souvent utilisé pour préciser qu'une fonction/méthode ne retourne pas de valeur (on attend du vide, donc rien)
- **Enum**: Un type de données permettant de créer son propre type de données customisé avec un choix prédéfini dans les valeurs (ex: enum Color {Red, Green, Blue}).
- **Array**: Un tableau pouvant contenir toutes sortes d'éléments (ex: [10,true,"google"]).
On peut également définir le type de données d'un tableau (ex: let tab:Array;)
- **Never**: Un type qui ne retourne jamais rien (boucle infinie ou erreur)
- **Null and Undefined**: Equivalent des types null et undefined Javascript.

Il en existe plein d'autres que vous pourrez retrouver sur la documentation en ligne du langage Typescript à www.typescriptlang.org

Templates Strings

Les "templates strings" permettent d'insérer plus facilement des valeurs de variables au sein d'une chaîne de caractère, ces dernières peuvent également être définies sur plusieurs lignes.

Il s'agit d'une nouveauté de la norme ES6, mais comme Typescript est un sur-ensemble de Javascript, elles sont supportées.

```
let jedi = {surname:"Obiwan", name:"Kenobi"};  
let msg = `${jedi.name} ${jedi.surname} is the Jedi Master`;  
console.log(msg);
```

Les variables

En Typescript, on n'emploie plus le mot clé **var** pour déclarer une variable mais le mot clé **let** et le mot clé **const**.

Le mot clé **const** est utilisé, comme son nom l'indique, pour déclarer une valeur **constante**, çàd une valeur ne pouvant être modifiée, si vous essayez de modifier une constante, une erreur est levée.

Le mot clé **let** est quand à lui utilisé pour déclarer une valeur variable, çàd une valeur capable de changer au cours de la durée de vie de votre programme.

Contrairement à Javascript, qui n'est pas typé, le Typescript l'est lui, comme son nom l'indique. Il est donc utile de préciser le type de données que l'on souhaite stocker au sein de la variable (ou de la constante) à l'aide de la syntaxe suivante:

```
// constante, ne peut être modifiée
const LIGHT_SPEED:string = "299 792 458 m / s";
// variable, peut être modifiée
let msg:string = "Hello World";
```

Les fonctions

En Typescript comme en Javascript, les fonctions sont incontournables. Il s'agit de blocs d'instructions répétables que l'on peut appeler (on dit invoquer) autant de fois que nécessaire.

Les fonctions Typescript, contrairement aux fonctions Javascript, précisent le type de données qu'elles renvoient, et si elles ne renvoient rien, alors le type de retour est **void** (du vide)

```
// les paramètres sont typés, et le type de la donnée
// retournée également à l'aide de la syntaxe suivante:
function sum( a:number, b:number ):number{
    return a + b;
}

// quand une fonction ne retourne aucune donnée
// alors on le précise en utilisant le type "void"
function notif( msg:string ):void{
    alert(msg);
}

// les fonctions anonymes sont également utilisables, comme en JS
let anonymous:Function = function():void{
    console.log("I am an anonymous function");
};

// on peut invoquer une fonction anonyme comme en JS
anonymous();
```

Les fonctions fléchées

En Typescript comme en Javascript ES6, les fonctions fléchées sont supportées, bien entendu le typage des paramètres et de la donnée de retour est à ajouter au sein de la version Typescript. Il est également possible de spécifier des valeurs par défaut aux paramètres. Les fonctions fléchées ont également l'avantage de préserver le contexte dans lequel elles sont déclarées.

```
// fonction fléchée anonyme avec valeur de paramètre par défaut
let hello = (param_user: string = "user"): void => {
  console.log("Hello", param_user);
};

hello(); // invoquons cette fonction

// les fonctions fléchées anonymes préservent le contexte
class Gandalf {
  name: string = "Gandalf";
  introduce = () => {
    console.log("Hello my name is", this.name);
  }
}

// testons notre code
new Gandalf().introduce();
```

Les boucles

En Typescript, les boucles fonctionnent comme en Javascript:

```
let i:number = 10;

// une boucle for classique en Typescript
for( i = 0; i < 10; i ++){
    console.log(i);
}

// une boucle while classique en Typescript
i = 10;
while( --i > -1 ){
    console.log(i);
}

// une boucle do while classique en Typescript
i = -1;
do{
    if( i == -1 ){
        i = 10;
    }
}while( --i > -1)
```

Les tableaux

TypeScript, comme JavaScript, permet de travailler avec des **tableaux** de valeurs. A la différence qu'en Typescript, les tableaux (si on le souhaite) peuvent être **typés**, ce qui signifie qu'ils ne peuvent contenir **qu'un seul type de données**. La déclaration de type pour les tableaux peut s'écrire de deux façons :

```
// première façon de déclarer un tableau typé
let notes:number[] = [
  0,10,12,7,8,20,13,15
];

// seconde façon de déclarer un tableau typé
let notes2:Array<number> = [
  0,10,12,7,8,20,13,15
];
```

Les enums

TypeScript nous permet de créer et d'utiliser des **enums**. Les enums permettent au développeur de créer un lot de **constantes** et de les regrouper de façon à former un nouveau **type de données**.

```
// créons un type de données "Direction"
// qui peut prendre 4 valeurs différentes
enum Direction { Up = 1, Down = 2, Left = 3, Right = 4 };

let haut: Direction = Direction.Up;
let bas: Direction = Direction.Down;
let gauche: Direction = Direction.Left;
let droite: Direction = Direction.Right;

console.log(haut, bas, gauche, droite);
```

Les Tuples

TypeScript nous permet de créer et d'utiliser des **tuples**. Les tuples permettent au développeur de créer des tableaux ordonnés avec des types prédéfinis.

```
// tableau fixe et ordonné  
let utilisateur: [string, number];  
utilisateur = ["Alice", 30]; // ✓  
utilisateur = [30, "Alice"]; // ✗ Ordre incorrect
```

Next-generation JavaScript

Impact sur la syntaxe

TypeScript est un sur-ensemble de JavaScript :

- Tout code JavaScript valide est aussi du TypeScript.
- On ajoute des annotations de types, des interfaces, des enums, etc.
- Ces ajouts n'existent plus dans le code JavaScript compilé.

```
// TypeScript
function addition(a: number, b: number): number {
  return a + b;
}

// JavaScript généré
function addition(a, b) {
  return a + b;
}
```

Conclusion : L'impact est surtout au niveau du développement et de la compilation, pas à l'exécution.

Automatiser la documentation

Le typage explicite permet de générer automatiquement une documentation à partir du code.

Voici un exemple avec **typedoc**

```
npm install typedoc --save-dev  
npx typedoc --entryPoints src/index.ts --out docs
```

```
/**  
 * Additionne deux nombres.  
 * @param a Premier nombre  
 * @param b Deuxième nombre  
 */  
function addition(a: number, b: number): number {  
  return a + b;  
}
```

Utiliser TypeScript avec du code JavaScript standard

On peut intégrer TypeScript dans un projet existant JavaScript :

1. Renommer petit à petit les fichiers .js en .ts ou .tsx
2. Activer allowJs dans tsconfig.json

```
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": true
  }
}

// JavaScript avec vérification TypeScript
/**
 * @param {number} a
 * @param {number} b
 */
function addition(a, b) {
  return a + b;
}
```

Inclure/générer des fichiers de définition pour la compatibilité

Les fichiers .d.ts contiennent uniquement des définitions de types, sans implémentation.

Inclure : pour utiliser une lib JS dans TS

```
npm install @types/express
```

Générer pour votre propre lib :

```
{  
  "compilerOptions": {  
    "declaration": true,  
    "outDir": "./dist"  
  }  
}
```

Bénéfice des getters/setters implicites

En TypeScript, on peut créer des accesseurs (get / set) qui :

1. Permettent de contrôler la lecture/écriture d'une propriété
2. Conservernt la même syntaxe que les attributs standards

```
class Utilisateur {  
  private _nom: string;  
  
  constructor(nom: string) {  
    this._nom = nom;  
  }  
  
  get nom(): string {  
    return this._nom.toUpperCase();  
  }  
  
  set nom(val: string) {  
    if (val.length < 2) throw new Error("Nom trop court");  
    this._nom = val;  
  }  
}  
  
const u = new Utilisateur("Alice");  
console.log(u.nom); // ALICE  
u.nom = "Bob";      // passe par le setter
```

Modules

A partir de ECMAScript 2015 (ES6), JavaScript introduit le concept de modules, et bien entendu, ce concept est supporté en Typescript

Les modules sont exécutés au sein de leur propre portée, et non pas au sein de la portée globale. Cela signifie que les variables, fonctions, classes, etc... Déclarées au sein d'un module ne sont pas visibles en dehors du module à moins qu'elles ne soient explicitement exportées à l'aide du mot-clé **export**. De fait, pour utiliser un élément exporté, au sein d'un module différent, il faut utiliser le mot-clé **import**.

```
export function toto(){  
  console.log("toto est beau");  
}  
  
import {toto} from './ts_modules_1';  
  
toto();
```

Namespaces

Utilisés pour regrouper du code sans système de modules

Moins courant aujourd'hui, mais utile pour code global ou libs anciennes

```
namespace Geometrie {  
  export function aireCercle(rayon: number): number {  
    return Math.PI * rayon * rayon;  
  }  
}  
  
console.log(Geometrie.aireCercle(10));
```

Typescript & POO

Programmation orientée objet (POO)

La programmation orientée objet est un style de programmation nous permettant de représenter des concepts informatiques sous forme d'objets. Pour conceptualiser un objet, nous avons besoin de plusieurs outils directement intégrés au langage. Il existe également plusieurs façons de "coder objet", Javascript est un langage **orienté objet par prototypage**, mais **Typescript** lui émule le comportement d'un **langage orienté objet par classe**. Il est important de retenir qu'un objet possède plusieurs caractéristiques:

- Un objet possède des **propriétés**, çàd des variables qui lui appartiennent en propre, la portée de ces propriétés peut être, publique, protégée ou privée.
- Un objet possède également des **méthodes**, çàd des fonctionnalités (fonctions) qui lui appartiennent en propre, la portée de ces méthodes peut être, publique, protégée ou privée.
- Dans un langage orienté objet par classe, un objet peut **hériter** d'un autre objet.
- Un objet peut prendre **plusieurs formes** et redéfinir ses propriétés héritées

Et il ne s'agit ici que des caractéristiques minimum, çàd celles qui définissent la base de la base d'un langage orienté objet, on parle des principes** d'encapsulation, d'héritage et de polymorphisme**.

Les classes

```
class Hero{
  // une propriété peut être publique, protégée ou privée
  public name:string;
  public power:string;

  // la fonction constructrice est invoquée automatiquement
  // à la création d'un nouvel objet
  constructor(
    param_name:string,
    param_power:string
  ){
    // on attribue à nos propriétés les valeurs passées
    // en paramètre
    this.name = param_name;
    this.power = param_power;
  }

  // une méthode peut être publique, protégée ou privée
  public sayMyName():void{
    console.log(this.name);
  }

  public sayMyPower():void{
    console.log(this.power);
  }
}

let myHero:Hero = new Hero("Batman", "Being rich");
myHero.sayMyName();
```

Le Singleton

Le design pattern Singleton permet de s'assurer qu'un objet n'est instancié qu'une seule fois à travers toute l'application. Il procure également un accès unifié à cette instance (ici via la méthode **getInstance**).

```
class MySingleton{
    private static _instance:MySingleton;
    public static getInstance():MySingleton{
        if( !MySingleton._instance ){
            MySingleton._instance = new MySingleton();
        }
        return MySingleton._instance;
    }

    private constructor(){}
}

// pour obtenir l'instance:
console.log( MySingleton.getInstance() == MySingleton.getInstance() );
```

Le pattern Factory

Centralise la création d'objets, utile quand la logique d'instanciation est complexe.

```
interface Vehicule {
  drive(): void;
}

class Voiture implements Vehicule {
  drive() { console.log("La voiture roule"); }
}

class Moto implements Vehicule {
  drive() { console.log("La moto roule"); }
}

class VehicleFactory {
  static create(type: "voiture" | "moto"): Vehicule {
    if (type === "voiture") return new Voiture();
    if (type === "moto") return new Moto();
    throw new Error("Type de véhicule inconnu");
  }
}

const v = VehicleFactory.create("voiture");
v.drive();
```

Interfaces & Duck Typing

Le Duck Typing : 'Si ça marche comme un canard et que ça cancanne, alors c'est un canard'. En TypeScript, une interface n'impose pas la classe exacte, juste la forme (structure) attendue.

```
interface Flying {  
  fly(): void;  
}  
  
function makeMeFly(obj:Flying) {  
  obj.fly();  
}  
  
// avec un objet  
const bird = { fly: () => console.log("L'oiseau vole") };  
  
// avec une classe  
  
class Boeing implements Flying{  
  fly(): void {  
    console.log("Le Boeing vole");  
  }  
}  
  
makeMeFly(bird); // ☒ Pas besoin que `bird` implémente explicitement `Flying`  
makeMeFly( new Boeing());
```

Héritage

```
class Personnage {  
    // on veut transmettre cette propriété à nos enfants, on utilise donc protected  
    protected name: string;  
    // on veut transmettre cette propriété à nos enfants, on utilise donc protected  
    protected lifepoint: number;  
  
    constructor(param_name: string, param_lifepoint: number) {  
        this.name = param_name;  
        this.lifepoint = param_lifepoint;  
    }  
  
    // on doit pouvoir demander à un personnage s'il est mort  
    // sans être soi-même un personnage, la portée est donc publique  
    public isDead(): boolean {  
        // si le nombre de points de vie est inférieur  
        // ou égal à 0 alors le personnage est mort  
        if (this.lifepoint <= 0) {  
            return true;  
        }  
        else {  
            return false;  
        }  
    }  
  
    public sayMyName():void{  
        console.log(this.name);  
    }  
}
```

Héritage

```
class Wizard extends Personnage{
  constructor( ){
    // on peut invoquer le constructor de la classe parente à l'aide de "super"
    super("Gandalf", 100);

    //... maintenant un nouvel objet de type wizard se nommera toujours
    // Gandalf et aura 100 points de vie
  }

  // on réécrit la méthode sayMyName défini par le parent
  // et héritée de celui-ci.
  public sayMyName():void{
    // mais on peut toujours invoquer l'ancienne "version" de la méthode
    // toujours à l'aide de "super"
    super.sayMyName();

    // si on veut, on peut ajouter des opérations supplémentaires
    console.log("I am a super magician !");
  }
}

// on crée un nouveau sorcier
let gandalf:Wizard = new Wizard();
// on crée un personnage Gollum qui possède 2 points de vie
let gollum:Personnage = new Personnage("Gollum", 2);

gandalf.sayMyName();
gollum.sayMyName();
```

Héritage multiple via Mixins

TypeScript n'autorise pas l'héritage multiple direct, mais on peut combiner plusieurs classes avec des mixins.

```
type Ctor<T = {}> = new (...args: any[]) => T;

function Warrior<TBase extends Ctor>(Base: TBase) {
  return class extends Base {
    fight() { console.log("Baston !"); }
  };
}

function GameCharacter<TBase extends Ctor>(Base: TBase) {
  return class extends Base {
    introduce() { console.log("Chui un barbare !"); }
  };
}

class CharacterBase {}
class Barbarian extends GameCharacter(Warrior(CharacterBase)) {}

const crom = new Barbarian();
crom.fight();
crom.introduce();
```

Advanced types

Gestion des types personnalisés

En TypeScript, on peut créer ses propres types pour :

- Simplifier le code
- Documenter l'intention
- Réutiliser une définition

```
type ID = string | number;
type Point = { x: number; y: number };

const identifiant: ID = 42;
const position: Point = { x: 10, y: 20 };

// ou

interface Utilisateur {
  nom: string;
  email: string;
}

const u: Utilisateur = { nom: "Alice", email: "alice@mail.com" };

// type est plus flexible (permet unions, intersections), interface est extensible.
```

Les génériques

Les génériques permettent de créer des structures ou fonctions qui fonctionnent avec n'importe quel type, tout en conservant la vérification.

```
// fonction générique
function identity<T>(value: T): T {
  return value;
}

identity<string>("Hello"); // T = string
identity(42); // T est inféré comme number

// classe générique
class Box<T> {
  private content: T;
  constructor(value: T) {
    this.content = value;
  }
  getContent(): T {
    return this.content;
  }
}

const box = new Box<number>(123);
console.log(box.getContent()); // number

const boiteTexte = new Box("Bonjour"); // T inféré comme string
```

Les restrictions des types génériques

On peut restreindre un générique avec extends pour n'accepter que certains types.

```
// Restriction à un type
function displayLength<T extends { length: number }>(valeur: T) {
  console.log(valeur.length);
}

displayLength("Hello"); // ☒ string a length
displayLength([1, 2, 3]); // ☒ tableau a length
displayLength(42); // ☐ number n'a pas length

// Restriction à une interface
interface Nameable { myName: string; }

function displayMyName<T extends Nameable>(obj: T) {
  console.log(obj.myName);
}

displayMyName({ myName: "Alice", age: 30 }); // ☒
```

Les décorateurs

Les décorateurs et les 'Metadata'

En TypeScript, un décorateur est une fonction spéciale qui peut être attachée à :

- Une classe
- Une propriété
- Une méthode
- Un paramètre

Important : il faut installer le package **reflect-metadata** et activer les décorateurs dans tsconfig.json :

- Les décorateurs enrichissent le code sans le modifier directement
- Les factories permettent de les paramétrer
- reflect-metadata sert à stocker et lire des données attachées à des éléments
- On peut cibler classes, propriétés, méthodes, paramètres
- Idéal pour des patterns avancés : IoC, validation, logging, sécurité

```
{
  "compilerOptions": {
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

```
import "reflect-metadata";
```

```
// target étant l'objet auquel vous souhaitez ajouter des metadata
Reflect.defineMetadata("role", "admin", target);
```

```
// target étant l'objet duquel vous souhaitez tirer des metadata
const role = Reflect.getMetadata("role", target);
```

Les décorateurs Factories

Un décorateur **simple** est une fonction, en revanche, un décorateur factory est une fonction qui retourne un décorateur, utile pour passer des paramètres.

```
function Logger(message: string) {  
  return function (constructor: Function) {  
    console.log(` ${message} - ${constructor.name} créé`);  
  };  
}  
  
@Logger("Création de la classe")  
class Test {}  
// Affiche : "Création de la classe - Test créé"
```

Les différents types de décorateurs

```
// décorateur de propriété
function Readonly(target: any, propertyKey: string) {
  Object.defineProperty(target, propertyKey, {writable: false});
}

// décorateur de paramètre
function LogParam(target: any, methodName: string, paramIndex: number) {
  console.log(`Paramètre #${paramIndex} de ${methodName}`);
}

// décorateur de fonction/méthode
function LogMethodCall(target: any, propertyKey: string, descriptor: PropertyDescriptor) {
  const func = descriptor.value;
  descriptor.value = function (...args: any[]) {
    console.log(`Appel de ${propertyKey} avec`, args);
    return func.apply(this, args);
  };
}

class TestClass {
  @Readonly
  nom = "Fixe";

  @LogMethodCall
  helloWorld( @LogParam param:string){ console.log(param)}
}

const e = new TestClass();
(e as any).nom = "Test"; // Ignoré en mode strict
```

Créer des décorateurs personnalisés

```
// Exemple avec reflect-metadata pour stocker des infos sur une méthode

import "reflect-metadata";

function Role(role: string) {
  return function (target: any, key?: string) {
    Reflect.defineMetadata("role", role, target, key!);
  };
}

class AdminService {
  @Role("admin")
  supprimer() {}
}

// Lecture
const role = Reflect.getMetadata("role", AdminService.prototype, "supprimer");
console.log(role); // "admin"
```

TypeScript en pratique

Usage avec Node.js

TypeScript s'intègre parfaitement avec Node.js pour du développement serveur.

```
# installation
npm init -y
npm install typescript ts-node @types/node --save-dev
npx tsc --init

# en développement
npx ts-node src/index.ts

# en production
npx tsc
node dist/index.js

# Utiliser ts-node-dev pour recharger automatiquement en développement.
npm install ts-node-dev --save-dev
npx ts-node-dev src/index.ts
```

Exemple de tsconfig pour node js:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true
  }
}
```

Livrer du code compatible JavaScript/Typescript

Si vous développez une librairie en TypeScript destinée à être utilisée par des projets JavaScript et TypeScript, il faut livrer :

Le Javascript compilé

Les définitions de types **.d.ts**

Exemple de configuration tsconfig.json pour une lib

```
{
  "compilerOptions": {
    "declaration": true,    // Génère les fichiers .d.ts
    "declarationMap": true, // Génère les maps pour navigation
    "outDir": "./dist",
    "target": "ES2019",
    "module": "commonjs",
    "esModuleInterop": true,
    "strict": true
  },
  "include": ["src/**/*"]
}
```

Arborescence du projet:

```
/src
  index.ts
/dist
  index.js    // Code JS compilé
  index.d.ts  // Définitions de types
```

Et on peut consommer la librairie normalement:

```
const { addition } = require("mylib");
console.log(addition(2, 3));
```